INDUSTRIAL PROJECT REPORT

ON

BVR Mission Data & Debriefing Dashboard

Submitted By

Name : Prathmesh Behal

Roll No. : 2915002723

Course : B.Tech. Computer Science Engineering

University : Maharaja Surajmal Institute Of Technology

Under the Supervision of

Name : Mr. Ghanshyam Meena

Designation : Scientist 'E'

Office : Institute for System Studies and Analyses (ISSA)

Duration of Internship : 15th June 2026 to 18th August 2026

Institute for Systems Studies and Analyses (ISSA)

Defence Research and Development Organisation (DRDO)

Ministry of Defence

Metcalfe House, Delhi-110054

Declaration by the candidate

I hereby declare that the project report entitled “BVR Mission Data & Debriefing Dashboard” is an original work carried out by me as part of my project/internship requirements for the degree of Bachelor of Technology in Computer Science Engineering at Maharaja Surajmal Institute of Technology.

The work presented in this report has been carried out by me during the course of the project under the guidance and supervision provided by the concerned authorities. The information, results, and observations presented in this report are based on the work undertaken during the project.

I further declare that this report has not been submitted, in whole or in part, to any other institution or university for the award of any degree, diploma, or other academic qualification. Wherever the work or ideas of others have been referred to, appropriate acknowledgement has been provided.

Acknowledgment

I would like to express my sincere gratitude to the **Defence Research and Development Organisation (DRDO)** and the Institute for Systems Studies and Analyses (ISSA) for providing me with the opportunity to undertake this project and gain valuable practical experience in a professional research environment.

I would like to express my deepest gratitude to my project mentor, **Mr.Ghanshyam Meena**, for their continuous guidance, technical support, valuable suggestions, and encouragement throughout the development of the BVR Mission Data & Debriefing Dashboard. Their guidance played an important role in helping me understand the project requirements and successfully implement the developed system.

I would also like to sincerely thank **Mr. Sunil Kumar Yadav** for their valuable guidance, technical assistance, feedback, and encouragement throughout the course of the project. Their support greatly contributed to my understanding of the project and its implementation.

I am also grateful to the faculty and administration of **Maharaja Surajmal Institute of Technology** for providing me with the academic foundation and support necessary to undertake and complete this project.

Finally, I would like to thank everyone who contributed directly or indirectly to the successful completion of this project.

Prathmesh Behal

About the Institute for System Studies and Analyses (ISSA)

The Institute for Systems Studies and Analyses (ISSA) is a premier laboratory of the Defence Research and Development Organisation (DRDO), located at Metcalfe House, Delhi. ISSA specialises in systems analysis, operational research, war-gaming, and modelling and simulation in support of the Indian Armed Forces. Its work spans the development of combat simulation environments, decision-support systems, and analytical tools that aid in the evaluation of weapon systems, tactics and force structures.

A significant thrust area of the laboratory is the high-fidelity simulation of air, land and naval engagements, where physics-based models of platforms and weapons are combined with behavioural models of combatants to study tactical outcomes. The present project—which couples a six-degree-of-freedom flight-dynamics simulation of beyond-visual-range air combat with modern reinforcement learning—falls squarely within this charter, contributing both a validated simulation environment and a data-driven tactical decision agent.

About the Defence Research and Development Organisation (DRDO)

The Defence Research and Development Organisation (DRDO) is the premier research and development wing of the Ministry of Defence, Government of India, with a vision to empower the nation with cutting-edge defence technologies and a mission to achieve self-reliance in critical defence systems. Established in 1958, DRDO today operates a nationwide network of laboratories engaged in diverse disciplines including aeronautics, armaments, combat vehicles, electronics, missiles, advanced computing and simulation, life sciences and naval systems.

DRDO has delivered a wide spectrum of strategic and tactical systems to the Indian Armed Forces and continues to advance indigenous capability across the technological frontier. Within this ecosystem, modelling, simulation and analysis play a central role in the design, evaluation and operational employment of complex systems—the domain to which this internship project contributes.

Contents

1. Introduction

1.1. Background and Motivation

Beyond Visual Range (BVR) air combat refers to engagements in which aircraft detect, track, and engage threats beyond the range of direct visual observation. Such missions involve multiple dynamic parameters, including aircraft position, altitude, velocity, heading, target information, missile activity, and mission outcomes. The large amount of information generated during a mission makes effective post-mission review important for understanding the sequence of events and evaluating the overall engagement.

In a simulation-based BVR environment, mission information is generated and stored during the execution of an engagement. However, reviewing this information directly from raw data can be difficult and time-consuming, particularly when the data contains multiple aircraft, events, and time-dependent parameters. A structured interface is therefore useful for presenting the available mission information in a clear and accessible form.

The **BVR Mission Data & Debriefing Dashboard** was developed to address this requirement. The dashboard provides a centralized web-based interface for viewing and organizing mission data generated by BVR simulations. It presents relevant mission information through structured tables, visual representations, mission details, and other interface components, allowing users to review the events and information associated with a completed mission more conveniently.

The project also focuses on making the debriefing process more efficient by reducing the need to manually inspect raw mission data. Features such as mission-wise data presentation and data export provide additional flexibility for reviewing and retaining mission information.

The overall motivation of the project is to develop a practical and user-friendly system that transforms available BVR mission data into an organized format suitable for post-mission debriefing and review.

1.2. Objectives

The primary objective of this project is to develop a web-based **BVR Mission Data & Debriefing Dashboard** that provides a structured and user-friendly interface for reviewing data generated from BVR simulation missions.

The major objectives of the project are:

- To develop a centralized dashboard for accessing and viewing BVR mission information.
- To organize mission data into a clear and easily understandable format.
- To provide users with relevant aircraft, missile, engagement, and mission information through the dashboard.

- To enable users to review completed missions and understand the sequence of events associated with an engagement.
- To provide suitable visual representations of mission information for easier interpretation.
- To implement data export functionality so that mission information can be saved in a structured CSV format for further use.
- To develop a reliable and responsive web interface that simplifies the post-mission debriefing process.

1.3. Project Contribution

The primary contribution of this project is the development of the **BVR Mission Data & Debriefing Dashboard**, a web-based system designed to provide a centralized interface for reviewing information generated from BVR simulation missions. The system organizes the available mission data and presents it through a structured and user-friendly interface, making it easier to review the details of completed missions.

The project includes the development and integration of frontend and backend components required to retrieve, process, and display mission information. The dashboard provides relevant aircraft, missile, engagement, and mission details in an organized manner, along with visual representations that assist in understanding the information during post-mission debriefing.

Another contribution of the project is the implementation of mission data export functionality. This allows the displayed mission information to be exported in CSV format, providing a convenient way to retain and use the data for further processing or offline reference. Overall, the project integrates data handling, backend APIs, frontend interface development, visualization, and export functionality into a single web-based debriefing system.

1.4. Organisation of the Report

This report is organised into several chapters covering the background, requirements, architecture, methodology, implementation, and evaluation of the **BVR Mission Data & Debriefing Dashboard**.

The first chapter introduces the project, its background and motivation, objectives, and major contributions. The second chapter presents the relevant background and literature related to BVR simulation, mission data, web-based visualization, and the technologies used in the project. The third chapter describes the platform, functional requirements, and technical requirements of the system.

The subsequent chapters explain the overall system architecture, the development methodology, and the implementation of the dashboard. The report then discusses the results obtained from the developed system along with its limitations and possible areas for future improvement. The final chapter presents the conclusion of the project, followed by supporting information and additional material in the appendix.

1.5. Project Directory Structure

The project follows a modular structure in which the frontend, backend, database, and supporting resources are organised into separate logical components. The overall directory structure of the project is presented below.

```
BVR Mission Data & Debriefing Dashboard/
|
├── Frontend/
|   ├── decision.html
|   ├── objectives.html
|   ├── doc15.html
|   ├── replay.html
|   ├── timeline.html
|   └── style15.css
|
├── Backend/
|   ├── server.js
|   └── index.js
|
├── Database/
|   └── database.sql
|
├── Map & Geographic Resources/
|   ├── custom.geo.json
|   ├── india-osm.geojson
|   └── map.jpg
|
├── Libraries & Resources/
|   ├── leaflet/
|   ├── v10.9.0-package/
|   └── Orbitron,Russo_One/
|
├── Configuration/
|   └── .vscode/
|
├── package.json
└── package-lock.json
```

2. Background and Literature Survey

2.1. BVR Mission Environment

Beyond Visual Range (BVR) refers to an air-combat scenario in which an aircraft engages or tracks another aircraft beyond the range of direct visual observation. Such engagements rely on information obtained from onboard sensors, communication systems, and other mission data to provide awareness of the surrounding environment.

A BVR mission can involve several types of information, including aircraft identification, position, altitude, velocity, heading, target information, weapon events, and mission outcomes. These parameters change throughout an engagement and collectively describe the progression of a mission.

In a simulation environment, this information can be generated and stored as mission data for subsequent review. The ability to access and organise this information is important for post-mission debriefing, as it allows users to review the events and states recorded during the mission without having to rely solely on the live simulation.

For this project, the BVR mission environment serves as the **source of the mission data presented by the dashboard**. The **BVR Mission Data & Debriefing Dashboard** does not perform the BVR simulation itself; instead, it provides an interface through which the available mission information can be accessed, organised, visualised, and reviewed.

2.2. Mission Data and Post-Mission Debriefing

A BVR mission generates a variety of information throughout its execution. This may include aircraft details, mission events, engagement information, missile-related events, timestamps, and the final outcome of the mission. Such information provides a record of the events that occurred during the mission and can be used for subsequent review.

Post-mission debriefing involves examining the recorded information after the completion of a mission. It allows users to review the sequence of events and obtain a consolidated view of the mission rather than relying on information observed during the live execution of the simulation.

When mission information is available in raw or structured data formats, reviewing it directly can be inconvenient, particularly when multiple types of information are involved. A dedicated debriefing interface can therefore organise this information into a more accessible format. Tables, visual elements, mission summaries, and other interface components can help users navigate the available information more efficiently.

The **BVR Mission Data & Debriefing Dashboard** is designed around this requirement. It provides a centralised interface through which available mission data can be accessed and presented in a structured manner, supporting the post-mission review process.

2.3. Mission Data Visualization

Mission data can contain a large amount of information that may be difficult to interpret when presented only in raw or tabular form. Data visualization provides a means of presenting such information in a more structured and understandable manner by representing relevant information through graphical and visual elements.

In the context of BVR mission debriefing, visualization can assist users in understanding mission events, comparing different parameters, and identifying important information more efficiently. Depending on the type of data, visual representations may include maps, timelines, charts, tables, and summary indicators.

A dashboard combines multiple forms of visualization within a single interface, allowing different aspects of a mission to be viewed without requiring users to examine separate data sources individually. This can improve accessibility and make the post-mission review process more organised.

The **BVR Mission Data & Debriefing Dashboard** applies these principles by presenting available mission information through a structured web interface. The dashboard brings together different mission-related data and visual components to provide a consolidated view of the selected mission, supporting easier navigation and review during the debriefing process.

2.4. Web-Based Dashboard Systems

Web-based dashboards provide a centralized interface for presenting information collected from different sources. Instead of requiring users to access and interpret data through separate files or applications, a dashboard can bring relevant information together within a single web-based environment.

A dashboard generally consists of multiple interface components that allow users to view, navigate, and interact with information. Depending on the application, these components may include tables, charts, maps, filters, summaries, and other interactive elements. Such an approach can improve accessibility and make large amounts of information easier to navigate.

For mission debriefing applications, a web-based dashboard can provide a convenient platform for reviewing completed missions and accessing different categories of mission information from a common interface. It also allows the presentation layer to be separated from the underlying data and application logic, making the system easier to maintain and extend.

The **BVR Mission Data & Debriefing Dashboard** follows this approach by providing a web-based interface for accessing and presenting mission information. The dashboard integrates the required interface components and backend services to provide users with a centralized environment for post-mission review.

2.5. Technologies Used

The **BVR Mission Data & Debriefing Dashboard** is developed as a web-based application using a combination of frontend and backend technologies. These technologies work together to provide the user interface, server-side processing, data management, and visualization functionality required by the system.

HTML is used to define the structure of the dashboard pages and organize the different interface components. **CSS** is used to control the appearance, layout, typography, and overall visual presentation of the dashboard.

JavaScript is used extensively to implement the interactive behaviour of the application. It handles user interactions, data processing on the client side, communication with backend services, and dynamic updating of dashboard components.

Node.js is used as the server-side runtime environment. It allows JavaScript to be executed on the backend and provides the foundation for handling server-side operations and communication between the dashboard and the data source.

Express.js is used as the web application framework for Node.js. It simplifies the development of server-side routes and APIs through which the frontend can request and receive mission-related information.

PostgreSQL is used as the relational database management system for storing and retrieving structured mission-related information. It provides the database layer through which the backend can access the required data for presentation on the dashboard.

Leaflet is used for map-based visualization within the dashboard. It provides the functionality required to display geographic information and interact with map elements.

The project also uses CSV export functionality is implemented to allow selected mission data to be saved in a structured format for further use or offline reference.

Together, these technologies form the technical foundation of the BVR Mission Data & Debriefing Dashboard, with the frontend responsible for presentation and interaction and the backend responsible for server-side processing and data access.

The selection of these technologies was based on the requirements of the dashboard and the need for effective communication between the user interface, application server, and database. The combination provides a modular development environment in which individual components can be developed and maintained independently while working together as a complete system. This technology stack also supports the interactive and data-driven nature of the dashboard, allowing mission information to be retrieved, processed, displayed, and exported through a single web-based application.

3. Platform Requirements and Specification

3.1. Development Environment

The development of the **BVR Mission Data & Debriefing Dashboard** was carried out in a web-based development environment consisting of frontend, backend, database, and supporting development tools. The system was developed and tested locally before being used as a complete dashboard application.

The frontend was developed using **HTML, CSS, and JavaScript**, which provide the structure, styling, and interactive functionality of the dashboard. The backend was developed using **Node.js and Express.js**, which handle server-side processing and provide APIs for communication between the dashboard and the database.

PostgreSQL was used as the relational database management system for storing and retrieving mission-related information. The application communicates with the database through the backend, allowing the required data to be retrieved and presented through the dashboard.

Visual Studio Code was used as the primary development environment for writing, organising, and managing the project files. A modern web browser was used to run and test the dashboard during development. **Leaflet** was used for the map-related visualization components of the application.

3.2. Software and Technology Requirements

The development and execution of the **BVR Mission Data & Debriefing Dashboard** require a combination of software tools and technologies that support the frontend, backend, database, and visualization components of the system.

The frontend requires a modern web browser capable of supporting HTML, CSS, and JavaScript. The backend requires **Node.js** as the runtime environment and **Express.js** for implementing the server-side application and APIs. The project dependencies are managed using **npm**, with the required packages and their versions specified through the project configuration files.

PostgreSQL is required for storing and retrieving the structured mission data used by the dashboard. The backend establishes communication with the PostgreSQL database to obtain the information required by the frontend.

For geographic visualization, the project uses **Leaflet**, along with the required map and geographic data resources. **Visual Studio Code** is used as the development environment for managing the source code and project files.

These software and technology requirements work together to provide the necessary environment for developing, running, and maintaining the BVR Mission Data & Debriefing Dashboard.

3.3. Functional Requirements

The functional requirements define the operations that the **BVR Mission Data & Debriefing Dashboard** is expected to perform. The system should provide users with a centralized interface for accessing and reviewing the available BVR mission information.

The dashboard should retrieve mission-related information from the backend and display it in an organized manner. Users should be able to access the relevant details of a selected mission, including available aircraft, engagement, missile, and mission information. The system should dynamically present the retrieved information through the appropriate dashboard components.

The system should provide map-based visualization where applicable, allowing geographic mission information to be displayed through the integrated map interface. It should also provide the necessary navigation and interaction features required to move between the different sections of the dashboard.

The dashboard should support post-mission review by presenting the available mission information in a structured format. It should also provide a mechanism for exporting mission information into **CSV format**, allowing users to retain the data for offline reference or further processing.

The backend should provide the required APIs for communication between the frontend and PostgreSQL database. The system should retrieve the requested information from the database and return it to the dashboard in a format that can be processed and displayed by the frontend.

3.4. Non-Functional Requirements

The non-functional requirements define the quality and operational characteristics expected from the **BVR Mission Data & Debriefing Dashboard**. These requirements ensure that the system is not only functional but also convenient, reliable, and maintainable.

The dashboard should provide a **user-friendly interface** through which mission information can be accessed without requiring users to interact directly with the underlying database or raw data. The information should be presented in a clear and organised manner so that users can navigate between different sections of the dashboard easily.

The system should provide **reliable communication** between the frontend, backend, and PostgreSQL database. Data retrieved from the database should be presented correctly, and the application should handle common errors without causing the entire dashboard to become unusable.

The dashboard should provide **reasonable response time** when retrieving and displaying mission information. The interface should remain responsive during normal usage, allowing users to navigate between pages and interact with available dashboard components smoothly.

The system should also be **maintainable and extensible**. Separating the frontend, backend, database, and supporting resources allows individual components to be modified or improved without requiring major changes to the complete application.

Finally, the dashboard should be **compatible with modern web browsers** and provide a consistent experience across the different pages and components of the application.

3.5. User Requirements

The primary user requirement of the **BVR Mission Data & Debriefing Dashboard** is to provide a simple and centralized interface for reviewing available BVR mission information. The user should be able to access the dashboard through a web browser without requiring direct interaction with the underlying database or backend services.

The user should be able to navigate between the different sections of the dashboard and access the information associated with the required mission. Mission-related details should be presented in a clear and organised manner so that the user can review the available information efficiently.

The dashboard should also allow the user to view relevant mission information through tables, visual elements, and map-based components where applicable. In addition, the user should be able to export mission data in CSV format when a copy of the information is required for further use or offline reference.

Overall, the system should minimise the complexity involved in accessing and reviewing mission data by providing the required information through a single, easy-to-use web interface.

4. System Architecture Overview

4.1. System Architecture

The **BVR Mission Data & Debriefing Dashboard** follows a client-server architecture in which the frontend, backend, and database work together to provide the required mission information to the user. The architecture separates the presentation layer from the server-side processing and data storage layers, allowing each component to perform a specific role within the system.

The **frontend layer** consists of the HTML pages, CSS, and JavaScript used to provide the dashboard interface. The different HTML pages represent the major sections of the dashboard, while JavaScript handles user interaction and requests the required mission information from the backend. The frontend also contains the map-related components used for displaying geographic information.

The **backend layer** is implemented using **Node.js and Express.js**. The server.js file acts as the main server application and defines the API endpoints used by the frontend. It receives requests from the dashboard, communicates with the PostgreSQL database, retrieves or updates the required information, and returns the resulting data to the frontend in JSON format.

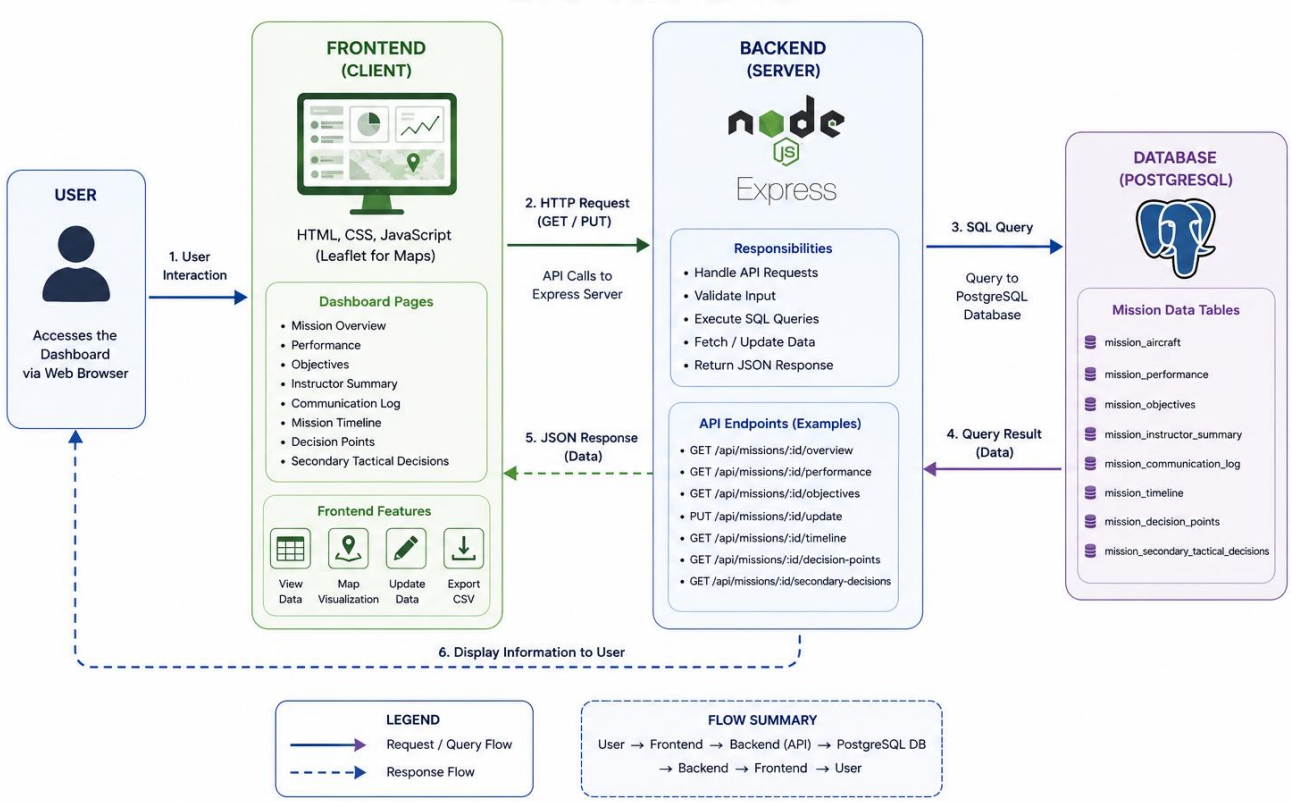
The **database layer** uses **PostgreSQL** to store the structured mission information. The database contains separate tables for different categories of information, including aircraft and

mission overview, performance, objectives, instructor summary, communication logs, mission timeline, decision points, and secondary tactical decisions.

The overall interaction between these layers begins when the user accesses or interacts with a dashboard page. The frontend sends a request to the appropriate backend API endpoint. The Express.js server processes the request and executes the corresponding SQL query through the PostgreSQL client. The retrieved data is then returned to the frontend, where JavaScript processes the response and updates the relevant elements of the dashboard.

The system also supports updating selected mission information. In such cases, the frontend sends an HTTP PUT request containing the updated values to the corresponding backend endpoint. The backend then executes the required SQL UPDATE operation in PostgreSQL and returns the result to the frontend.

Thus, the overall architecture can be represented as:



The response follows the reverse path, with the retrieved information travelling from PostgreSQL through the backend API to the frontend, where it is presented to the user through the dashboard.

4.2. Frontend Architecture

The frontend of the **BVR Mission Data & Debriefing Dashboard** is responsible for providing the user interface through which mission information can be viewed and interacted with. It is implemented using **HTML, CSS, and JavaScript**, with **Leaflet** integrated for map-based visualization.

The dashboard is divided into multiple HTML pages, with each page serving a specific purpose in the debriefing process. The **Overview** page provides the main mission information, while the **Objectives**, **Timeline**, **Replay**, and **Decision** pages provide access to different categories of mission-related information. A common navigation header is used across the pages to allow users to move between these sections.

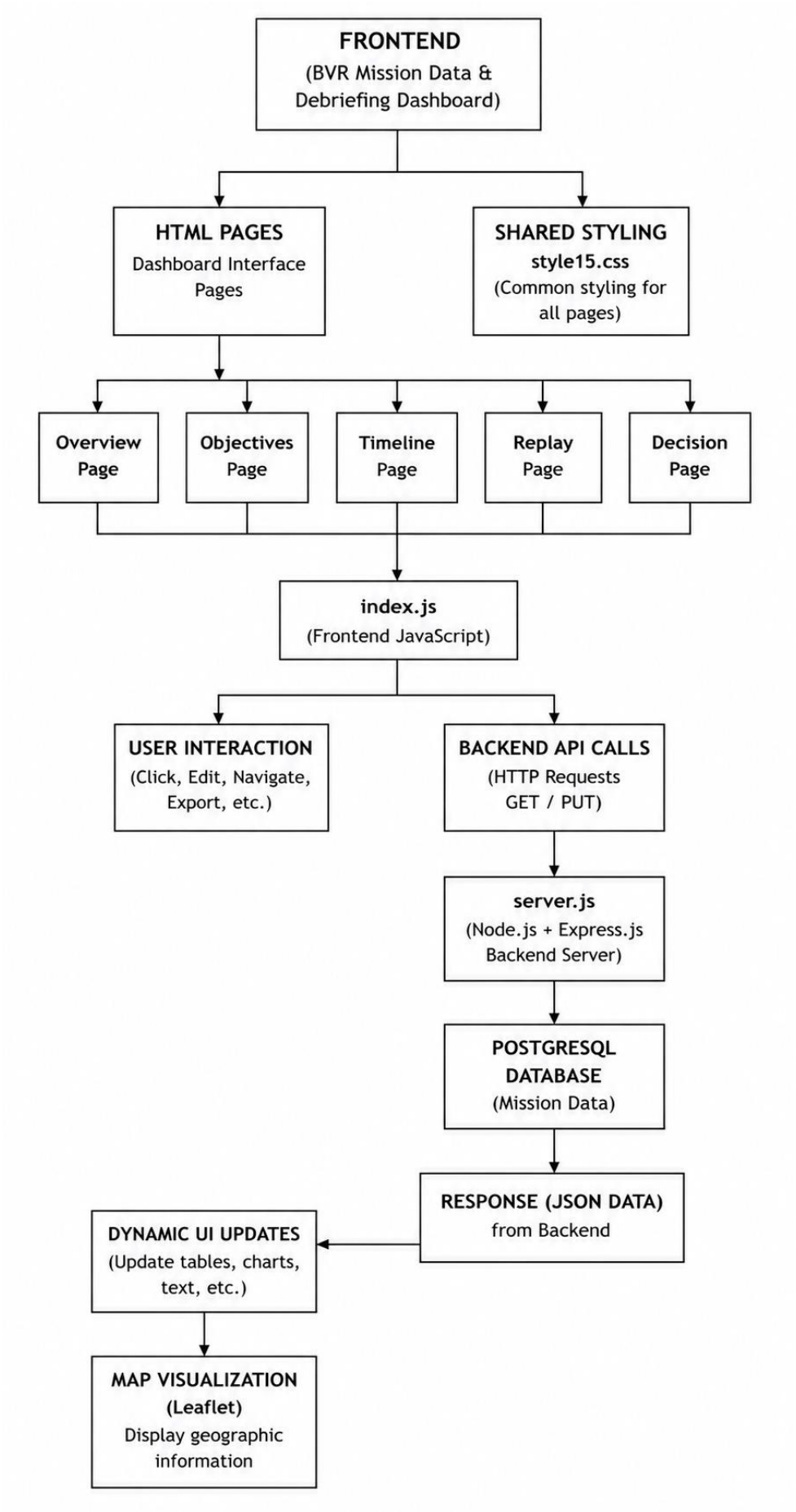
The style15.css file provides the common visual styling for the dashboard. It defines the layout, colours, typography, navigation elements, panels, tables, buttons, and other interface components. A locally stored Orbitron font is also used as part of the visual design of the dashboard.

The index.js file contains the main client-side JavaScript logic shared across the dashboard pages. It manages user interactions, communicates with the backend using HTTP requests, processes the returned data, and dynamically updates the corresponding elements of the interface. The JavaScript also handles functions such as editing selected mission information and exporting mission data to CSV format.

The **Leaflet** library is used on pages that require map visualization. It provides the functionality required to create and control interactive maps and display geographic information using the available map and geographic data resources.

The frontend therefore acts as the presentation and interaction layer of the system. It receives the required information from the backend, processes it through JavaScript, and presents the resulting mission information to the user through the different dashboard pages.

The overall frontend structure can therefore be represented as:


FRONTEND
(BVR Mission Data & Debriefing Dashboard)
HTML PAGES
Dashboard Interface Pages
SHARED STYLING
style15.css
(Common styling for all pages)
Overview Page
Objectives Page
Timeline Page
Replay Page
Decision Page
index.js
(Frontend JavaScript)
USER INTERACTION
(Click, Edit, Navigate, Export, etc.)
BACKEND API CALLS
(HTTP Requests GET / PUT)
server.js
(Node.js + Express.js Backend Server)
POSTGRESQL DATABASE
(Mission Data)
RESPONSE (JSON DATA)
from Backend
DYNAMIC UI UPDATES
(Update tables, charts, text, etc.)
MAP VISUALIZATION
(Leaflet)
Display geographic information

4.3. Backend Architecture

The backend of the **BVR Mission Data & Debriefing Dashboard** is implemented using **Node.js and Express.js** and is responsible for handling communication between the frontend and the PostgreSQL database. The backend acts as an intermediate layer, receiving requests from the frontend, processing them, interacting with the database, and returning the required information.

The main backend implementation is contained in the server.js file. The application uses Express.js to create the web server and define the API routes required by the dashboard. These routes allow the frontend to request specific mission information and, where required, submit updated information to the database.

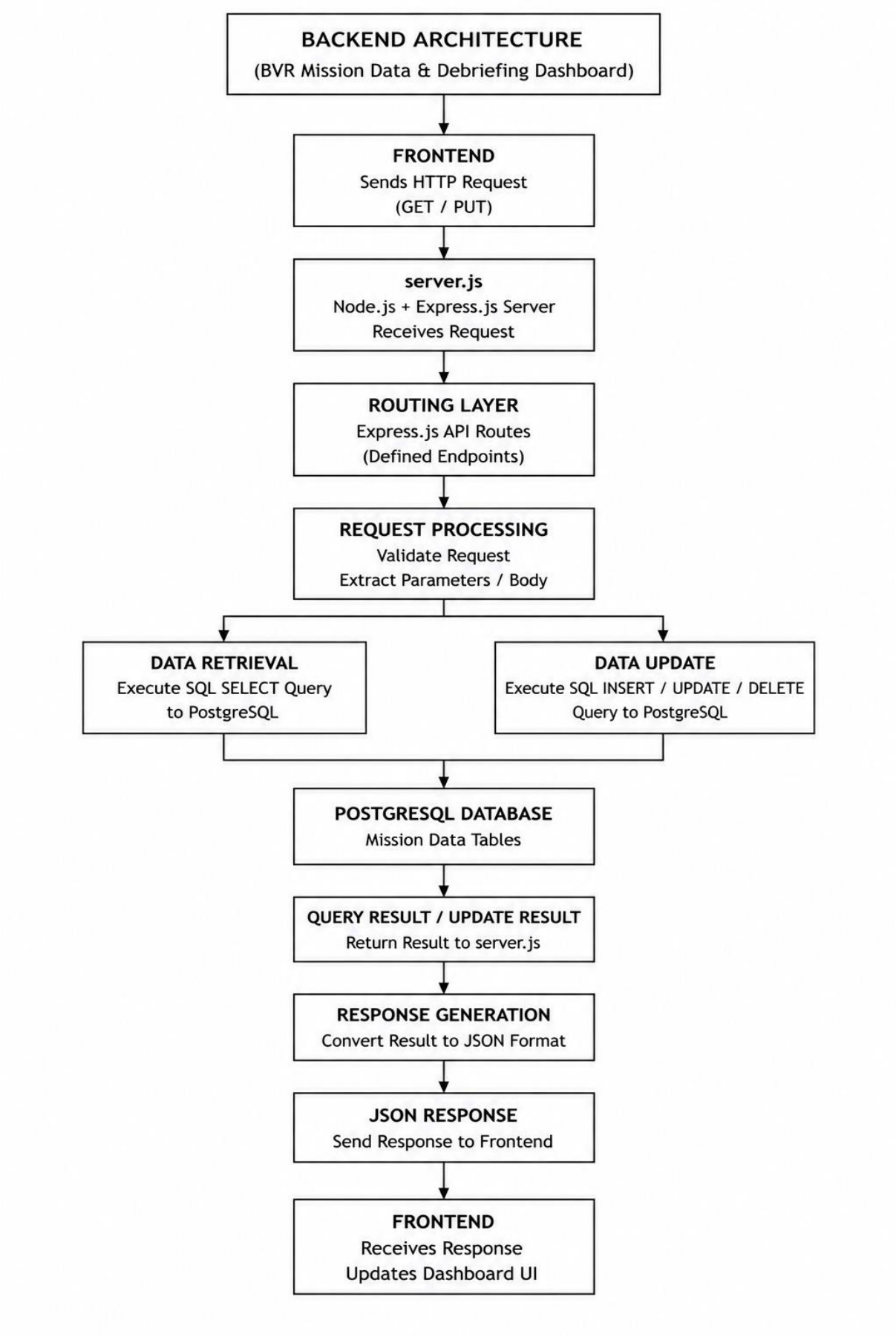
When a user accesses a particular section of the dashboard, the frontend JavaScript sends an HTTP request to the appropriate API endpoint. The backend receives the request and determines the required operation. For data retrieval, it executes the corresponding SQL query against the PostgreSQL database and obtains the requested mission information.

The retrieved database records are then processed by the backend and returned to the frontend in **JSON format**. The frontend JavaScript uses this response to populate the appropriate tables, text fields, visual components, or other elements of the dashboard.

The backend also supports updating mission information. When an editable value is modified through the dashboard, the frontend sends a PUT request to the corresponding API endpoint. The backend processes the submitted information and performs the required update operation in PostgreSQL before returning the result to the frontend.

This architecture keeps database operations within the backend rather than exposing the database directly to the user interface. It provides a clear separation between the presentation layer and data layer while allowing the dashboard to retrieve and update mission information through defined APIs.

The overall backend flow can be represented as:


BACKEND ARCHITECTURE
(BVR Mission Data & Debriefing Dashboard)
FRONTEND
Sends HTTP Request
(GET / PUT)
server.js
Node.js + Express.js Server
Receives Request
ROUTING LAYER
Express.js API Routes
(Defined Endpoints)
REQUEST PROCESSING
Validate Request
Extract Parameters / Body
DATA RETRIEVAL
Execute SQL SELECT Query
to PostgreSQL
DATA UPDATE
Execute SQL INSERT / UPDATE / DELETE
Query to PostgreSQL
POSTGRESQL DATABASE
Mission Data Tables
QUERY RESULT / UPDATE RESULT
Return Result to server.js
RESPONSE GENERATION
Convert Result to JSON Format
JSON RESPONSE
Send Response to Frontend
FRONTEND
Receives Response
Updates Dashboard UI

4.4. Database Architecture

The database layer of the **BVR Mission Data & Debriefing Dashboard** is implemented using **PostgreSQL**, which provides persistent storage for the structured mission information used by the application. The database separates the storage of mission data from the frontend and backend components, allowing the information to be retrieved whenever it is required by the dashboard.

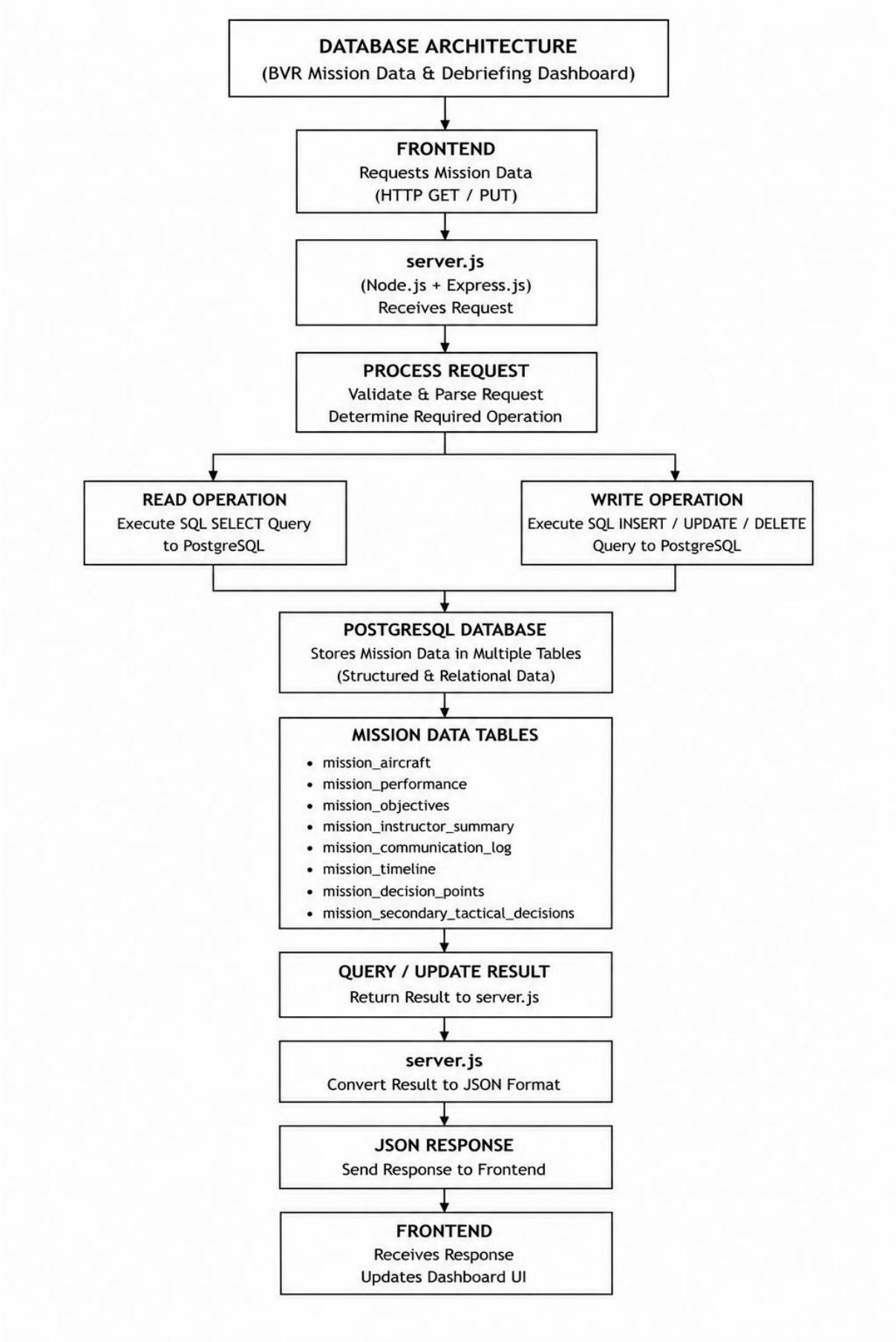
The mission information is organised into multiple tables, with each table representing a particular category of information associated with a mission. These include aircraft and mission overview information, performance data, objectives, instructor summary, communication logs, mission timeline, decision points, and secondary tactical decisions. This organisation allows different types of mission information to be stored and accessed independently.

The backend communicates with PostgreSQL using database queries. When the frontend requests information, the request is passed to the appropriate API endpoint in server.js. The backend then executes the required SQL query against the relevant database table and retrieves the requested records.

For operations that modify mission information, the backend executes the corresponding SQL update operation after receiving the updated values from the frontend. The result of the database operation is then returned to the frontend through the API.

The use of PostgreSQL provides a structured and persistent data layer for the dashboard. By keeping database operations within the backend, the system maintains a separation between the user interface and the underlying data storage, while providing controlled access to the mission information required by the application.

The simplified database interaction can be represented as:


DATABASE ARCHITECTURE
(BVR Mission Data & Debriefing Dashboard)
FRONTEND
Requests Mission Data
(HTTP GET / PUT)
server.js
(Node.js + Express.js)
Receives Request
PROCESS REQUEST
Validate & Parse Request
Determine Required Operation
READ OPERATION
Execute SQL SELECT Query
to PostgreSQL
WRITE OPERATION
Execute SQL INSERT / UPDATE / DELETE
Query to PostgreSQL
POSTGRESQL DATABASE
Stores Mission Data in Multiple Tables
(Structured & Relational Data)
MISSION DATA TABLES
mission_aircraft
mission_performance
mission_objectives
mission_instructor_summary
mission_communication_log
mission_timeline
mission_decision_points
mission_secondary_tactical_decisions
QUERY / UPDATE RESULT
Return Result to server.js
server.js
Convert Result to JSON Format
JSON RESPONSE
Send Response to Frontend
FRONTEND
Receives Response
Updates Dashboard UI

4.5. Data Flow

The **BVR Mission Data & Debriefing Dashboard** follows a request-response based data flow between the frontend, backend, and PostgreSQL database. The frontend acts as the interaction layer, while the backend manages the communication with the database and returns the required information to the dashboard.

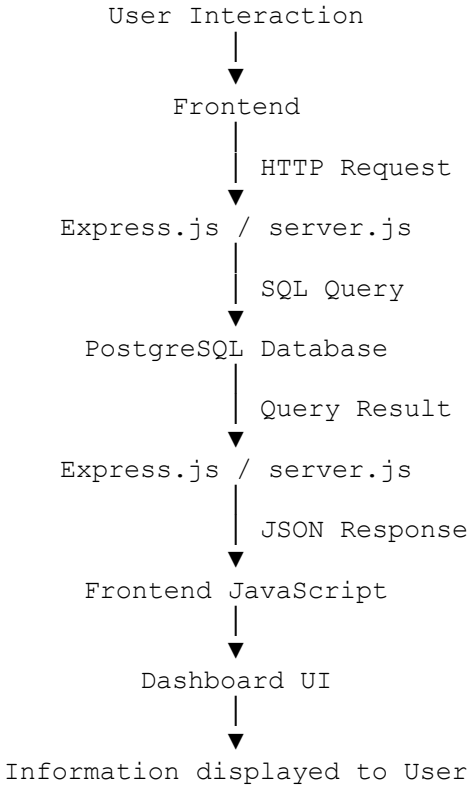
The process begins when the user interacts with a dashboard page or selects a particular mission. The frontend JavaScript identifies the required information and sends an HTTP request to the appropriate API endpoint provided by the backend.

The request is received by `server.js`, which uses Express.js to process the request and determine the required database operation. For data retrieval, the backend constructs and executes the corresponding SQL query against PostgreSQL. The database processes the query and returns the requested mission records to the backend.

The backend then prepares the retrieved information as a JSON response and sends it back to the frontend. The JavaScript running in the browser processes this response and dynamically updates the relevant dashboard elements, such as tables, text fields, timelines, or other visual components.

For operations that modify existing information, the process follows a similar flow. The updated information is submitted from the frontend using an HTTP `PUT` request. The backend processes the request and performs the required database update before returning the operation result to the frontend.

The overall data flow can therefore be summarised as:



This request-response flow provides a clear separation between the user interface, application logic, and database layer. It also allows the dashboard to retrieve and present mission information dynamically without requiring the user to access the database directly.

4.6. Map and Visualization Components

The **BVR Mission Data & Debriefing Dashboard** includes map-based visualization to provide a geographical representation of relevant mission information. The map component is implemented using the **Leaflet** JavaScript library, which provides the functionality required to create and interact with the map within the web interface.

The dashboard uses geographic resources such as GeoJSON data to provide the required geographical information for the map. These resources allow geographical features to be represented within the dashboard and provide a visual context for the mission-related information being displayed.

The map is integrated with the frontend JavaScript so that it can be initialized and updated as required by the dashboard. User interaction with the map is handled through the frontend, allowing the visualization to form part of the overall mission debriefing interface.

In addition to the map, the dashboard uses other visual components such as structured tables, information panels, timelines, and mission summaries. These components are used to organize different categories of mission information and make the available data easier to navigate and review.

The combination of map-based visualization and structured interface components allows the dashboard to present mission information in a more accessible manner while keeping the different aspects of the debriefing process within a single web-based interface.

5. Dashboard Design and Functional Modules

5.1. Dashboard Overview

The **BVR Mission Data & Debriefing Dashboard** is a web-based application developed to provide a centralized platform for reviewing and presenting information associated with completed BVR missions. The primary purpose of the dashboard is to bring together different categories of mission information in a structured interface so that users can access the required information during the post-mission debriefing process.

The dashboard follows a modular design in which different aspects of a mission are presented through dedicated sections. Instead of presenting all available information on a single screen, the system separates the information into logically organised pages. This reduces visual complexity and allows users to focus on a particular category of information when reviewing a mission.

The main dashboard provides access to the different sections of the application through a common navigation interface. The available sections include mission overview information, objectives, performance-related information, instructor summary, communication logs, mission timeline, decision points, secondary tactical decisions, and mission replay. Each section is designed to present a specific type of mission information while maintaining a consistent visual structure throughout the application.

The dashboard operates using a client-server model. When a user selects or requests mission information, the frontend JavaScript sends a request to the appropriate backend API. The backend processes the request and retrieves the required information from the PostgreSQL database. The returned data is then provided to the frontend in JSON format, where it is processed and dynamically displayed within the relevant dashboard components.

The interface uses a combination of tables, information panels, textual fields, navigation elements, timelines, and map-based visualization to present the retrieved information. This allows different forms of structured mission data to be viewed within a common environment. The use of visual and tabular components also makes it easier to locate specific information without directly interacting with the underlying database.

An important aspect of the dashboard is its ability to support the review of a mission as a complete sequence rather than treating individual data records independently. Information from different sections can be accessed through the dashboard navigation, allowing the user to examine mission details, review objectives and decisions, inspect communication and timeline information, and access the available replay and map-related features.

The dashboard also includes interactive functionality for selected data fields. Where editing is supported, users can modify the permitted information through the interface, after which the updated values are sent to the backend and stored in the PostgreSQL database. This provides a controlled method of maintaining the information presented by the dashboard.

In addition to on-screen presentation, the system provides a **CSV export facility** for mission information. The export functionality converts the selected mission data into a structured CSV file, allowing the information to be retained for offline reference or used for further processing outside the dashboard.

The overall design therefore focuses on centralization, accessibility, and structured presentation of mission information. By integrating data retrieval, visualization, navigation, editing, and export capabilities within a single web application, the dashboard provides a unified environment for post-mission review and debriefing.

5.2. Mission Overview Module

The Mission Overview module serves as the primary entry point for reviewing the information associated with a selected BVR mission. It provides a consolidated view of the key mission details and gives the user an initial understanding of the mission before moving to the more specific sections of the dashboard.

The module presents the available mission information in a structured format rather than requiring the user to inspect individual database records. Important details associated with the mission are displayed through dedicated interface elements, allowing information to be located and reviewed efficiently.

The overview also acts as a central navigation point for the remaining sections of the dashboard. From the mission overview, the user can access additional information related to objectives, performance, communication, timeline, decisions, and replay. This organisation allows the user to move from a general understanding of the mission to more detailed aspects of the debriefing process.

The information displayed in the module is retrieved from the backend through the appropriate API endpoints. The frontend processes the JSON response received from the server and dynamically populates the relevant fields on the page. This ensures that the information shown on the dashboard corresponds to the selected mission stored in the PostgreSQL database.

The Mission Overview module therefore provides the initial consolidated representation of a mission and establishes the context for the detailed review performed through the other modules of the dashboard.

5.3. Objectives and Performance Modules

The Objectives and Performance modules provide detailed information about the mission goals and the recorded performance-related information associated with the selected mission. These modules allow the user to move beyond the basic mission overview and examine specific aspects of the mission during the debriefing process.

The **Objectives module** presents the objectives associated with the mission in a structured format. It allows users to review the intended goals and requirements associated with the mission and provides a dedicated section for examining objective-related information. Presenting the objectives separately helps establish a clear reference point when reviewing the other mission information.

The **Performance module** presents the available performance-related information associated with the mission. The information is retrieved from the backend and displayed through the

dashboard interface, allowing the user to review the recorded values without directly accessing the database.

Both modules follow the same client-server data flow used throughout the dashboard. When the corresponding page is accessed, the frontend requests the required information from the backend API. The backend retrieves the relevant records from PostgreSQL and returns them to the frontend in JSON format. The frontend then processes the response and dynamically updates the appropriate interface elements.

The separation of objectives and performance information from the general mission overview allows the dashboard to present mission details at different levels of depth. Users can first obtain an overall understanding of the mission and then access these modules when more specific information is required during the debriefing process.

5.4. Communication Log and Mission Timeline

The Communication Log and Mission Timeline modules provide a chronological view of information associated with the mission. These modules are intended to help the user review the progression of events and communication activities that occurred during the mission.

The **Communication Log** presents the available communication-related records in an organised format. The information can include recorded communication events and their associated details, allowing users to review the communication information relevant to the selected mission during the debriefing process.

The **Mission Timeline** provides a chronological representation of recorded mission events. By arranging events according to their associated time or sequence, the module allows users to follow the progression of the mission and understand how different events occurred relative to one another.

Both modules retrieve their information from the backend through dedicated API requests. When the corresponding page is accessed, the frontend requests the required records from the backend. The backend executes the appropriate database queries against PostgreSQL and returns the resulting information in JSON format. The frontend then processes the response and presents the information through the relevant interface components.

The chronological presentation provided by the Mission Timeline complements the information available in other dashboard modules. For example, information related to objectives, decisions, or mission outcomes can be reviewed alongside the timeline to provide a clearer understanding of when particular events occurred.

Together, the Communication Log and Mission Timeline provide an organised view of the temporal aspects of a mission, supporting a more systematic review of events during the post-mission debriefing process.

5.5. Decision and Tactical Information Modules

The Decision and Tactical Information modules provide a dedicated view of decision-related information associated with the selected mission. These modules allow users to review the decisions recorded during the mission and examine the additional tactical information available for the debriefing process.

The **Decision Points** module presents the recorded decision points associated with the mission. It provides the relevant information in a structured format so that users can review individual decisions and their associated details. This helps organise decision-related information separately from the general mission overview and timeline.

The **Secondary Tactical Decisions** module provides access to additional tactical decision information recorded for the mission. By presenting this information through a dedicated interface, the dashboard allows users to examine tactical details that may not be part of the primary decision-point information.

The information for both modules is retrieved through the backend API and stored in the PostgreSQL database. When the relevant page is accessed, the frontend sends a request to the corresponding API endpoint. The backend processes the request, retrieves the required records from PostgreSQL, and returns the information to the frontend in JSON format.

The frontend then processes the returned data and dynamically displays the information within the appropriate interface components. This approach keeps the decision-related information organised while allowing it to be accessed alongside the other mission debriefing modules.

Key functions of the Decision and Tactical Information Modules

- **Decision Point Review:** Provides a structured view of the recorded decision points associated with the mission, allowing users to review individual decisions and their corresponding information.
- **Tactical Decision Review:** Presents secondary tactical decisions separately so that additional decision-making information can be examined without mixing it with the primary decision points.
- **Mission Context:** Allows decision-related information to be considered alongside other mission information, such as objectives, timeline, and performance.
- **Structured Presentation:** Organises decision-related records into readable interface components, making individual entries easier to locate and review.
- **Database Integration:** Retrieves the required decision and tactical information from PostgreSQL through the backend API rather than accessing the database directly from the frontend.
- **Dynamic Display:** Processes the JSON data returned by the backend and dynamically updates the relevant dashboard components.

.

5.6. Mission Replay and Map Visualization

The Mission Replay and Map Visualization module provides a visual representation of mission-related information and supports the review of the mission through an interactive interface. It complements the tabular and textual information presented in the other dashboard modules by providing a geographical and sequential view of the available mission data.

The **Mission Replay** component allows the user to review the available mission information in a replay-oriented interface. It provides a visual means of following the mission information and examining the events associated with the selected mission. This provides an additional method of reviewing the mission beyond static tables and text-based information.

The **map visualization** is implemented using the **Leaflet** JavaScript library. Leaflet provides the functionality required to create an interactive map within the dashboard and display geographic information using the available map resources. The project includes geographic data in formats such as GeoJSON, which can be used by the frontend to represent geographical features within the map.

The map component is integrated with the frontend JavaScript and forms part of the overall dashboard interface. The frontend manages the initialization and interaction with the map and can use the available mission information to present relevant geographical details.

The visual representation provided by the replay and map components is particularly useful when reviewing information that has a spatial or sequential relationship. Instead of examining individual records independently, the user can use the visual interface to obtain a more connected view of the available mission information.

The module therefore complements the other debriefing sections by providing:

- **Interactive Map:** Provides a geographical representation using the Leaflet mapping library.
- **Mission Replay:** Provides a dedicated interface for reviewing mission-related information in a replay-oriented manner.
- **Geographic Context:** Places available geographic information within a visual map environment.
- **Interactive Navigation:** Allows the user to interact with the visual representation rather than relying only on static data.
- **Integration with Mission Data:** Works alongside the other dashboard modules to provide additional context during post-mission review.

Together, the replay and map components enhance the dashboard by providing a visual method of reviewing mission information and complementing the structured data presented throughout the other modules.

5.7. Data Export Functionally

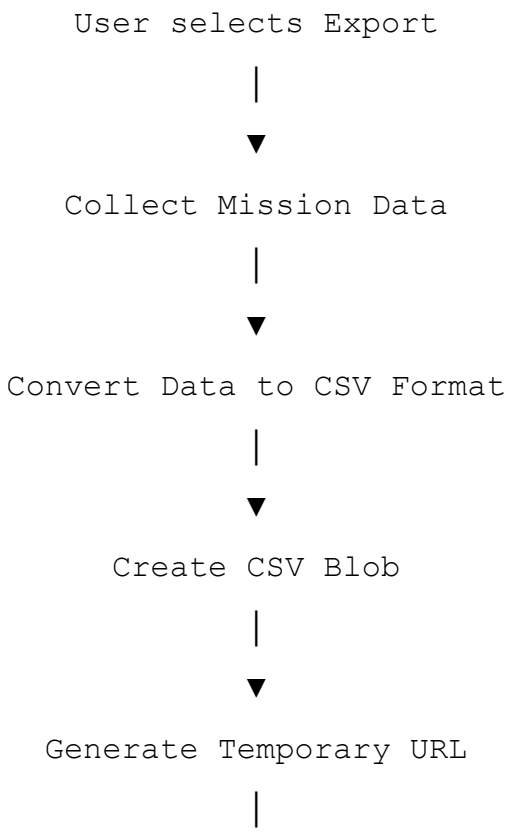
The **BVR Mission Data & Debriefing Dashboard** provides a CSV export functionality that allows users to save mission information displayed by the dashboard in a structured and portable format. This feature provides an alternative to viewing the information only within the web interface and allows the exported data to be retained for offline reference or further processing.

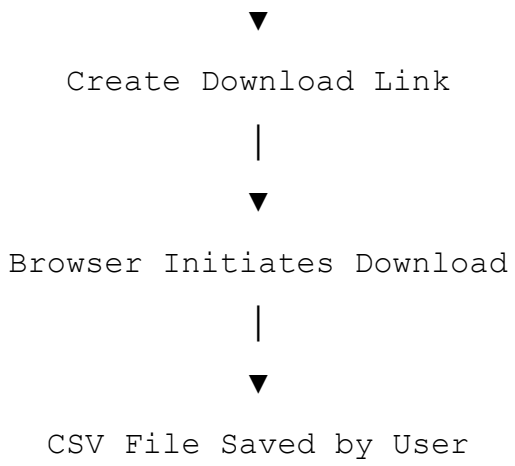
The export functionality is implemented using JavaScript on the frontend. When the user initiates an export, the application gathers the relevant mission information and converts the values into a CSV-formatted string. The data is organised into appropriate fields and records so that the resulting file can be opened using spreadsheet applications or other software capable of processing CSV files.

Special handling is applied to values that may contain characters such as commas or quotation marks. This ensures that individual values remain correctly separated within the generated CSV file and prevents the exported data from becoming incorrectly formatted.

Once the CSV content has been generated, the application creates a **Blob** containing the file data. A temporary object URL is then generated from the Blob and associated with a dynamically created download link. The browser uses this link to initiate the file download, after which the temporary resources are released.

The overall export process can be represented as:





The export functionality therefore provides a simple mechanism for converting the mission information available within the dashboard into a reusable file. It extends the dashboard beyond on-screen debriefing by allowing mission data to be retained and accessed independently of the web application.

6. Implementation and Engineering Details

6.1. Frontend Implementation

The frontend of the **BVR Mission Data & Debriefing Dashboard** is implemented using **HTML, CSS, and JavaScript**. It forms the presentation layer of the application and is responsible for displaying mission information, handling user interactions, communicating with the backend APIs, and dynamically updating the interface.

The dashboard is divided into multiple HTML pages, with each page corresponding to a particular area of the debriefing process. These pages include the mission overview, objectives, performance, instructor summary, communication log, mission timeline, decision points, secondary tactical decisions, and replay-related interfaces. This page-based organisation allows different categories of mission information to be presented separately while maintaining a consistent navigation structure.

The visual appearance and layout of the dashboard are controlled through CSS. The style15.css file defines common interface elements such as navigation components, panels, tables, buttons, typography, spacing, and page layouts. The styling provides consistency across the different dashboard pages and ensures that the information is presented in a structured manner.

The main client-side logic is implemented using JavaScript. The index.js file contains functions responsible for handling user interactions, retrieving mission information from the backend, processing API responses, updating HTML elements dynamically, and implementing additional dashboard functionality. JavaScript allows the interface to update according to the data received from the server without requiring the user to manually modify the underlying data.

The frontend communicates with the backend through HTTP requests to the defined API endpoints. When information is required, JavaScript sends a request to the backend and waits for the JSON response. Once the response is received, the returned data is processed and inserted into the appropriate elements of the webpage. This approach allows the same interface to display different mission information depending on the selected data.

The frontend also implements interactive functionality for selected data fields. Where editing is supported, the user can modify the relevant information through the dashboard. The JavaScript collects the updated values and sends them to the appropriate backend endpoint, allowing the changes to be stored in the database.

In addition to data retrieval and interaction, the frontend contains the logic required for **CSV export**. The application collects the required mission information, converts it into CSV format, creates a downloadable Blob, and triggers the browser's download mechanism. This allows users to obtain a local copy of the mission information without directly accessing the database.

The frontend also incorporates **Leaflet** for map-based visualization. JavaScript is used to initialize the map, load the required geographic resources, and manage the interactive map elements within the dashboard.

Overall, the frontend implementation acts as the main interaction layer of the system. It combines the HTML structure, CSS presentation, JavaScript functionality, API

communication, CSV export, and map visualization into a unified interface through which users can review and interact with BVR mission information. The modular structure of the frontend allows individual pages and components to perform specific functions while maintaining a consistent interface throughout the application. The use of JavaScript for dynamic data handling also ensures that information retrieved from the backend can be displayed and updated without requiring changes to the underlying HTML structure.

The frontend further provides a clear separation between presentation and data processing. The interface is responsible for presenting information and handling user interactions, while database operations remain within the backend. This separation makes the application easier to maintain and allows changes to individual interface components without directly affecting the database layer. The frontend therefore serves as the primary point of interaction between the user and the complete BVR Mission Data & Debriefing Dashboard, bringing together the different functional modules into a single web-based environment.

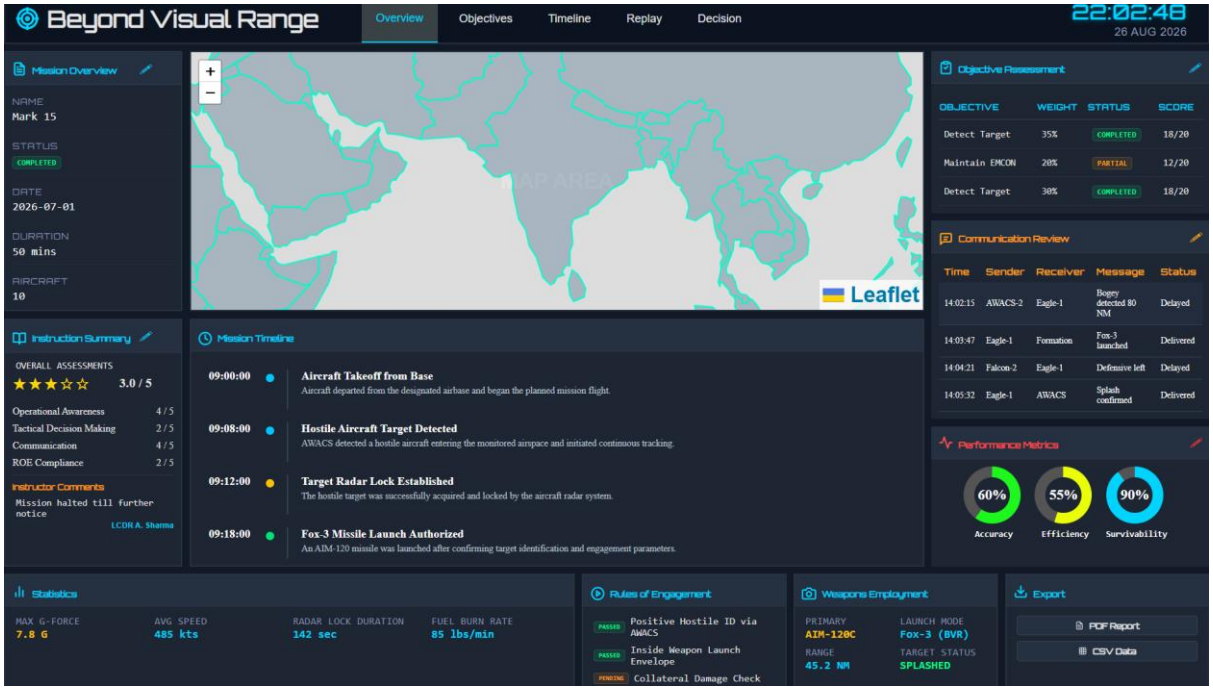


Figure 6.1: *Frontend interface of the BVR Mission Data & Debriefing Dashboard.*

6.2. Backend Implementation

The backend of the **BVR Mission Data & Debriefing Dashboard** is implemented using **Node.js** and **Express.js**. It provides the server-side layer responsible for handling requests from the frontend, communicating with the PostgreSQL database, processing the retrieved information, and returning the required results to the dashboard.

The primary backend implementation is contained in the server.js file. Node.js provides the runtime environment in which the server-side JavaScript code is executed, while Express.js is used to create the web server and define the API endpoints. These endpoints act as the communication interface between the frontend pages and the database.

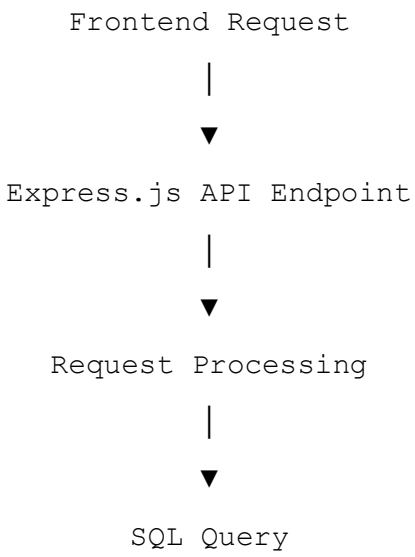
When the dashboard requires information, the frontend sends an HTTP request to the appropriate API endpoint. Express receives the request and executes the corresponding server-side function. Depending on the requested operation, the function performs a database query, processes the result, and prepares the information for transmission back to the frontend.

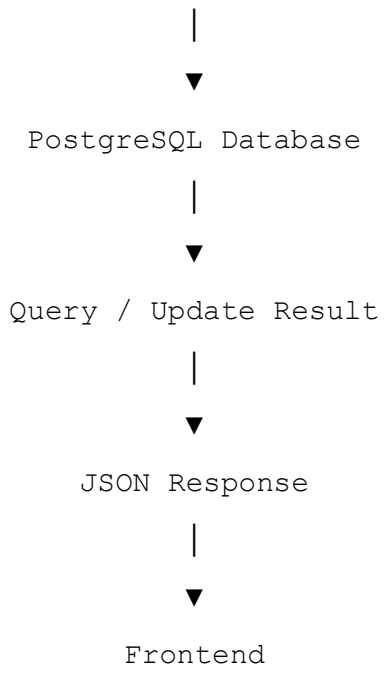
The backend uses SQL queries to retrieve the required records from PostgreSQL. Different API endpoints are used for different categories of mission information, allowing the application to request only the data required by a particular dashboard section. This approach avoids unnecessary retrieval of unrelated information and keeps the interaction between the frontend and database organised.

After the database operation is completed, the backend returns the retrieved information to the frontend in **JSON format**. JSON provides a structured representation of the database records that can be easily processed by JavaScript. The frontend receives this response and uses the returned values to dynamically populate the appropriate dashboard elements.

The backend also implements functionality for updating selected mission information. When an editable value is changed by the user, the frontend sends an HTTP PUT request containing the updated information. The corresponding Express route receives the request and executes the required SQL UPDATE operation against PostgreSQL. After the operation is completed, the backend returns a response indicating the result of the request.

The backend therefore follows a clear request-processing-response cycle:





An important advantage of this architecture is that the frontend does not directly access the PostgreSQL database. All database communication is handled by the backend, which provides a controlled layer between the user interface and the stored mission information. This separation also makes the application easier to maintain because changes to database queries or server-side processing can be made within the backend without requiring direct database logic in every frontend page.

The backend also provides a foundation for extending the dashboard in the future. Additional API endpoints can be introduced when new mission data categories or operations are required, while the existing frontend-backend communication model can remain unchanged. This makes the backend an important component in maintaining the overall modularity and scalability of the dashboard.

6.3. Database Implementation

The database component of the **BVR Mission Data & Debriefing Dashboard** is implemented using **PostgreSQL**, a relational database management system used to store and manage the structured information required by the application. The database provides persistent storage, allowing mission information to remain available after the completion of a mission and to be retrieved whenever it is required by the dashboard.

The database structure is defined using SQL and contains multiple tables corresponding to different categories of mission information. This organisation allows information to be stored according to its purpose while keeping the database structured and accessible to the backend application.

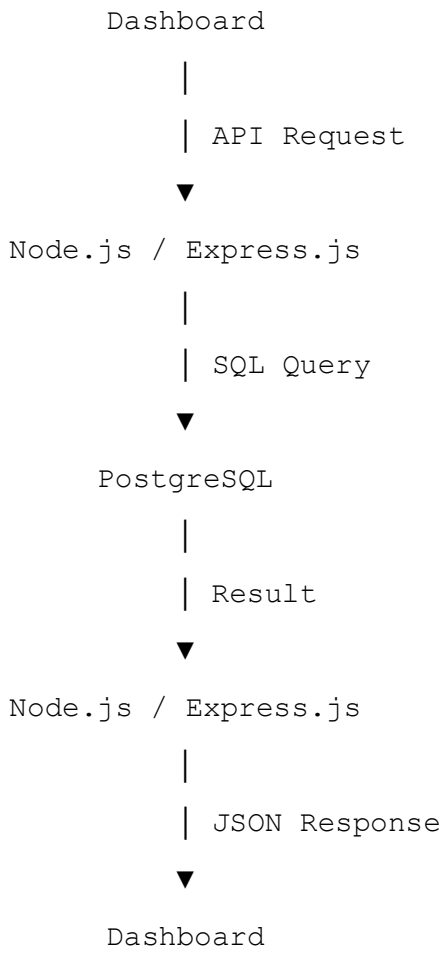
The database contains information related to different aspects of a mission, including the mission overview, aircraft information, performance-related data, objectives, instructor summary, communication logs, mission timeline, decision points, and secondary tactical

decisions. Separating these categories into appropriate database structures allows the dashboard to retrieve specific information according to the page or module being accessed.

The PostgreSQL database is not accessed directly by the frontend. Instead, the **Node.js and Express.js backend** acts as the intermediary layer. When the frontend requests information, the backend receives the request and executes the appropriate SQL query against PostgreSQL. The resulting records are then processed by the backend and returned to the frontend through the corresponding API endpoint.

The same approach is used when information needs to be modified. The frontend sends the updated values to the backend through an HTTP request, and the backend executes the required SQL UPDATE operation against the relevant database record. This ensures that database operations remain controlled through the server-side application.

The database interaction can be represented as:



The use of PostgreSQL provides several advantages for the project. As a relational database, it allows mission information to be stored in a structured format and queried using SQL. It also provides persistent storage, making the information available across different dashboard sessions. The separation of the database layer from the frontend further improves the organisation of the application and prevents direct exposure of the database to users.

The database implementation therefore forms the persistent data layer of the dashboard. It works together with the backend APIs to provide the frontend with the mission information required by the different debriefing modules while also supporting controlled modification of editable information.

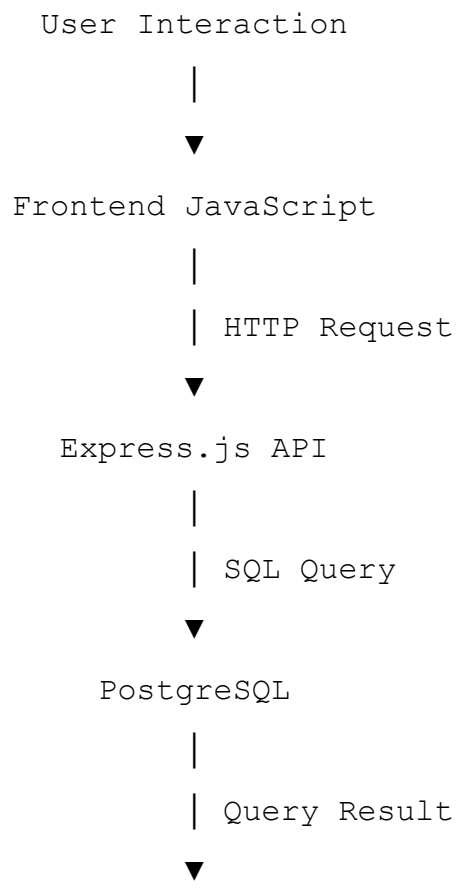
6.4. Frontend-Backend Communication

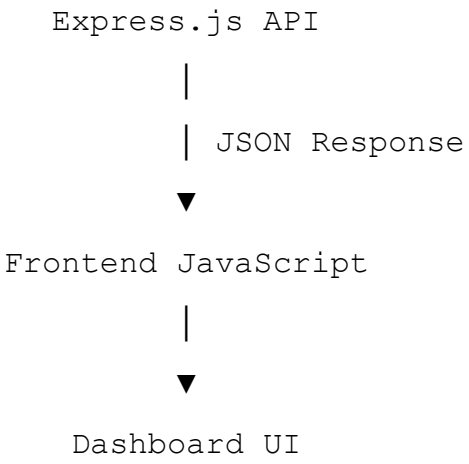
The communication between the frontend and backend forms an essential part of the **BVR Mission Data & Debriefing Dashboard**. It allows the user interface to request mission information from the server without directly accessing the PostgreSQL database. The communication is primarily implemented using HTTP requests between the frontend JavaScript and the Express.js API endpoints.

When a dashboard page requires mission information, the frontend JavaScript sends a request to the appropriate backend endpoint. The request specifies the operation or information required by the particular dashboard module. The Express.js server receives the request and executes the corresponding server-side logic.

For data retrieval operations, the backend performs the required SQL query against PostgreSQL. Once the database returns the requested records, the backend converts the result into a JSON response and sends it back to the frontend. JavaScript then processes the response and uses the returned values to populate the relevant elements of the dashboard.

The communication process can be represented as:





The same communication mechanism is used when the user performs an operation that modifies supported mission information. In such cases, the frontend sends an HTTP PUT request containing the updated values. The backend receives the request, performs the required database update, and returns a response indicating the result of the operation.

The use of JSON as the communication format provides a structured way of transferring information between the backend and frontend. The returned data can contain multiple fields and records while remaining easy for JavaScript to process. This allows the same communication mechanism to support different dashboard modules and different categories of mission information.

The frontend therefore remains responsible for presentation and user interaction, while the backend manages request processing and database communication. This separation prevents database operations from being performed directly within the browser and provides a clear communication layer between the user interface and the data storage system.

The overall request-response mechanism enables the dashboard to dynamically retrieve and display mission information while maintaining a structured separation between the frontend, application logic, and database.

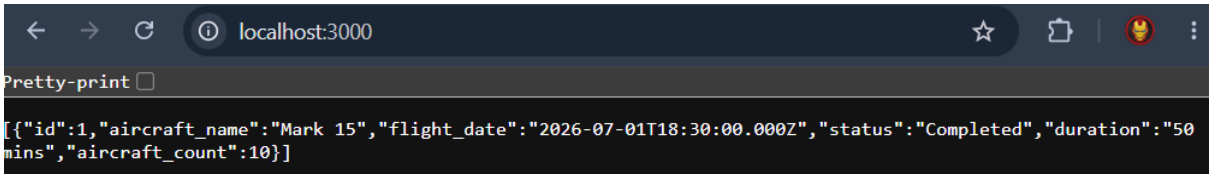


Figure 6.4: *JSON response returned by the backend API*

6.5. Data Retrieval and Rendering

The **BVR Mission Data & Debriefing Dashboard** retrieves mission information from the PostgreSQL database through the backend API and dynamically renders the received data on the frontend. This process allows the dashboard to display database records without storing the actual mission values directly within the HTML pages.

When a dashboard page requires information, the JavaScript code sends an HTTP request to the corresponding API endpoint. The backend processes the request, executes the required SQL query, and returns the result as a **JSON response**. The frontend then receives this response and converts it into a format that can be accessed through JavaScript.

For example, an API response may contain information in the following JSON format:


```
[
  {
    "id": 1,
    "aircraft_name": "Mark 15",
    "flight_date": "2026-07-01T18:30:00.000Z",
    "status": "Completed",
    "duration": "50 mins",
    "aircraft_count": 10
  }
]
```


After the response is received, JavaScript can access individual values using their corresponding keys. For example, the aircraft name can be accessed using:

data[0].aircraft_name

Here, data represents the JavaScript array containing the returned object, data[0] refers to the first object in the array, and aircraft_name accesses the corresponding value.

The retrieved values are then inserted into the appropriate HTML elements of the dashboard. This process is commonly performed by selecting an element and changing its content through JavaScript. As a result, the webpage is updated dynamically based on the information received from the backend.

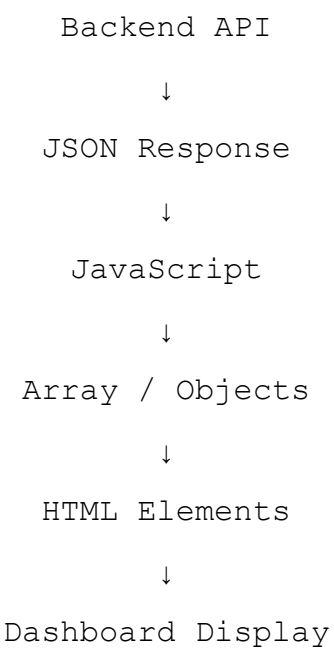
The overall process can be represented as:

PostgreSQL Database

↓

SQL Query

↓



This approach separates **data storage, data processing, and data presentation**. PostgreSQL is responsible for storing the information, the backend is responsible for retrieving and delivering it, and the frontend JavaScript is responsible for processing and presenting it to the user.

Dynamic rendering also allows the same dashboard interface to display different mission records without creating separate HTML pages for each mission. The page structure remains the same while the displayed values are populated according to the data returned by the API.

This data retrieval and rendering mechanism is therefore central to the dashboard, as it connects the stored mission information with the visual interface presented to the user.

6.6. Data Editing and Database Updates

The **BVR Mission Data & Debriefing Dashboard** provides editing functionality for selected mission information, allowing permitted values to be modified directly through the dashboard interface. This avoids the need for users to manually modify records within the database.

When an editable field is selected, the frontend provides an appropriate interface through which the user can enter or modify the required value. Once the user confirms the change, JavaScript collects the updated value and prepares it for transmission to the backend.

The updated information is sent to the appropriate Express.js API endpoint using an HTTP **PUT request**. The request contains the information required by the backend to identify the relevant record and the new value that needs to be stored.

The backend receives the request and processes the submitted information. It then executes the corresponding SQL **UPDATE** operation against the PostgreSQL database. Once the database operation is completed, the backend returns a response to the frontend indicating the result of the operation.

The overall update process can be represented as:

```
User edits a value
        ↓
Frontend JavaScript
        ↓
  HTTP PUT Request
        ↓
   Express.js API
        ↓
  SQL UPDATE Query
        ↓
PostgreSQL Database
        ↓
   Update Result
        ↓
   JSON Response
        ↓
Updated Dashboard Value
```

This process ensures that the value displayed on the dashboard and the value stored in the database remain synchronised after a successful update. The frontend is responsible for

collecting the user's input and displaying the result, while the backend controls the actual database modification.

The editing functionality also follows the separation of responsibilities used throughout the application. The user interacts only with the dashboard interface, the frontend communicates with the API, and the backend performs the database operation. This prevents direct database access from the browser and provides a controlled mechanism for modifying mission information.

The update mechanism therefore extends the dashboard beyond simple data visualization by allowing selected mission information to be maintained through the same web-based interface used for post-mission review.

6.7. CSV Export Implementation

The **BVR Mission Data & Debriefing Dashboard** includes a CSV export feature that allows mission information to be downloaded from the web application in a structured file format. The functionality is implemented on the frontend using JavaScript and does not require the user to access the PostgreSQL database directly.

When the user selects the export option, the JavaScript code first collects the relevant mission information available on the dashboard. The data is organised into rows and columns so that it can be represented in the standard comma-separated values format.

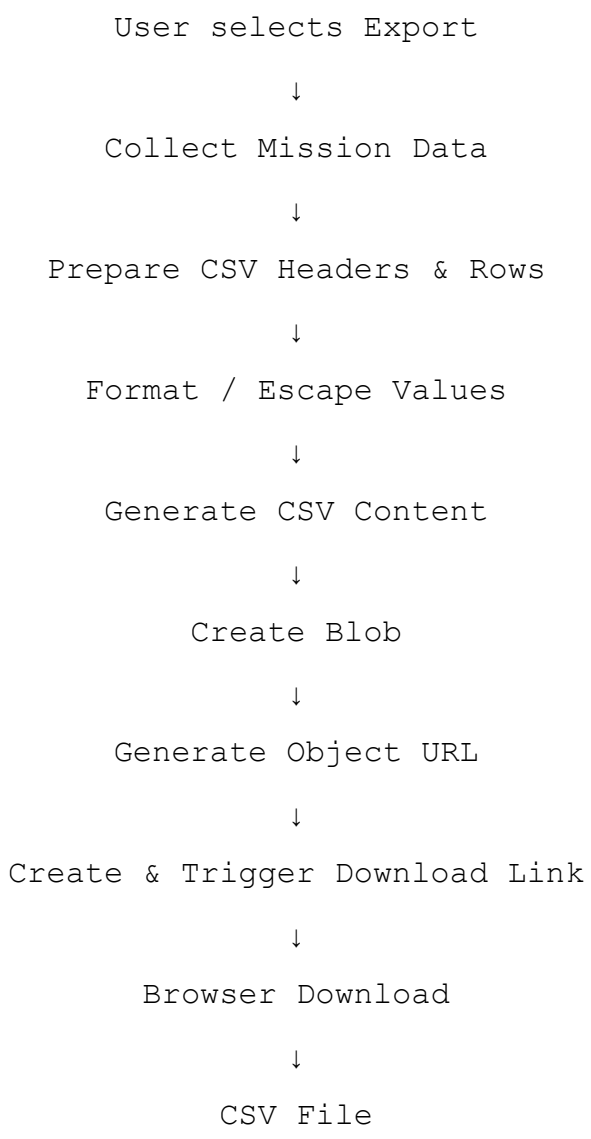
Before generating the file, the application processes individual values to ensure that special characters such as commas, quotation marks, or line breaks do not interfere with the CSV structure. Values that require additional formatting are enclosed or escaped appropriately so that the resulting file can be interpreted correctly by spreadsheet applications and other CSV-compatible software.

The generated CSV follows a tabular structure in which each column represents a particular field and each row represents a corresponding mission record. This structure makes the exported file easy to read and allows the same information displayed within the dashboard to be transferred into external tools without significant changes to its organisation. The export process therefore preserves the relationship between the different mission fields while converting the information into a standard format.

After the CSV content has been prepared, the application creates a **Blob** containing the generated text. A Blob is a browser object used to represent data as a file-like object. In this case, it allows the JavaScript application to treat the generated CSV content as a downloadable file.

The browser then generates a temporary object URL for the Blob. JavaScript creates a temporary download link, assigns the generated URL to it, and programmatically triggers the link. This causes the browser's normal download mechanism to save the generated CSV file to the user's system.

The complete export process can be represented as:



Once the download has been initiated, the temporary object URL is no longer required and can be released by the application. This helps prevent unnecessary browser resources from being retained after the file has been generated.

The CSV export mechanism provides a convenient way of transferring mission information from the dashboard into a commonly supported data format. The exported file can subsequently be opened using spreadsheet software or processed by other applications, making the mission information available beyond the dashboard interface.

The implementation therefore provides an additional way of accessing mission information while keeping the export process simple for the user. It also demonstrates how client-side JavaScript can generate and download a file dynamically without requiring a separate file-generation service on the backend.

The CSV export functionality therefore provides a practical method for taking the mission information presented within the dashboard and converting it into a reusable external file. It

allows the user to retain a copy of the mission data independently of the dashboard and provides flexibility for further analysis, documentation, or sharing. The implementation demonstrates how client-side JavaScript can generate a structured data file and use the browser's built-in download mechanism to make the exported information directly available to the user.

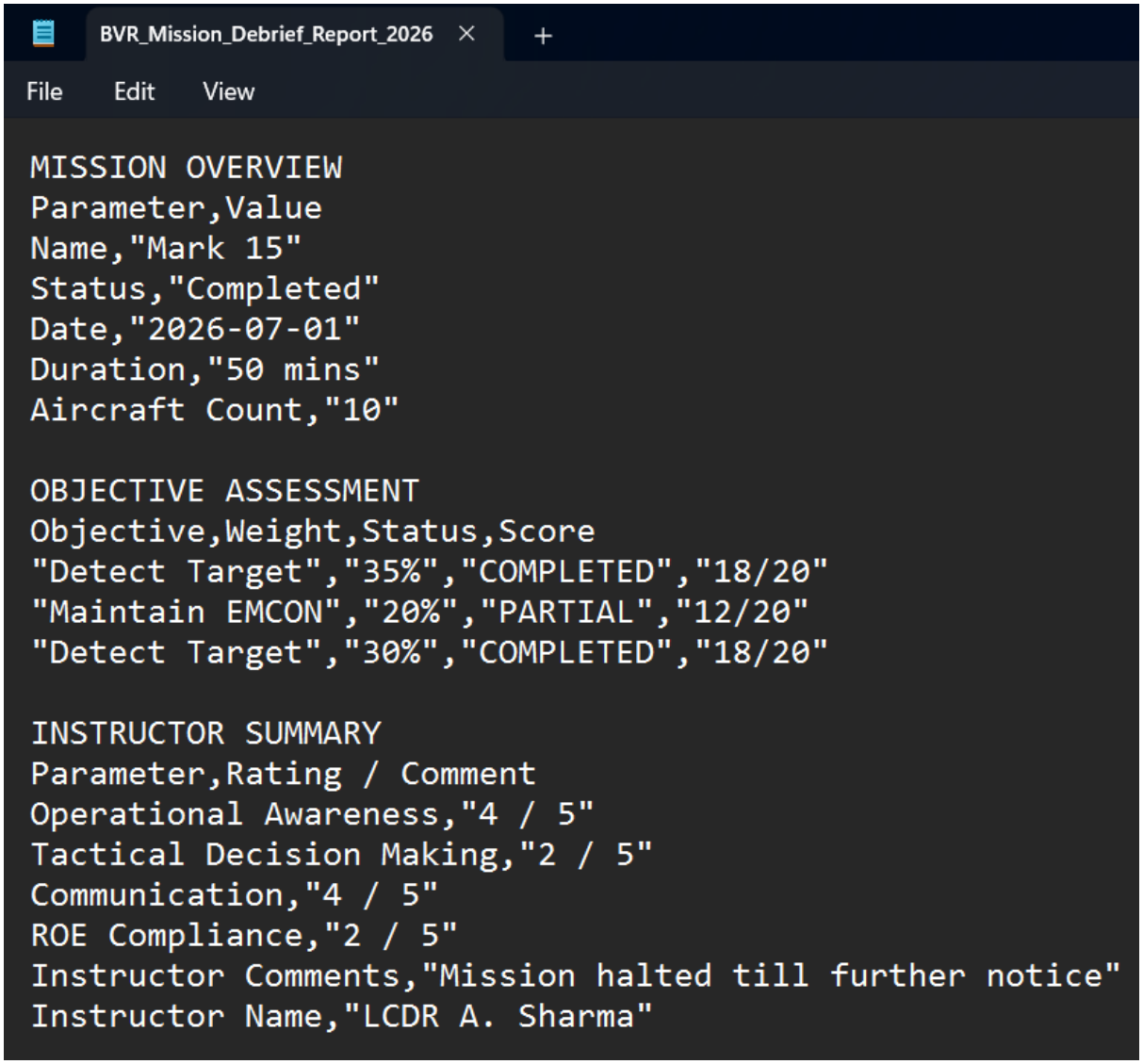


Figure 6.7: Exported mission data in CSV format

6.8. Map Implementation

The **BVR Mission Data & Debriefing Dashboard** incorporates an interactive map component to provide geographical visualization within the application. The map is implemented using the **Leaflet JavaScript library**, which provides the required functionality for creating, displaying, and interacting with map elements in a web browser.

Leaflet is integrated into the frontend through JavaScript. The map is initialized within the relevant dashboard page, where the required map container is defined in the HTML structure. JavaScript then creates the Leaflet map and configures the required geographic layers and map settings.

The project uses geographic data resources, including **GeoJSON files**, to represent geographical information within the map. GeoJSON provides a structured format for representing geographic features such as points, lines, and boundaries. These resources can be loaded by the frontend and displayed as layers within the Leaflet map.

The map implementation follows the general frontend data flow of the application. The JavaScript code controls the map, loads the required geographic resources, and manages the visual elements displayed to the user. This allows the map to operate as an integrated component of the dashboard rather than as a separate application.

The basic map implementation can be represented as:

HTML Map Container

↓

Frontend JavaScript

↓

Initialize Leaflet Map

↓

Load Geographic Data

↓

GeoJSON / Map Resources

↓

Create Map Layers

↓

Display Geographic Information

↓

User Interaction

The map provides an additional visual dimension to the dashboard by allowing geographical information to be viewed spatially. This complements the tables, timelines, and textual information presented in the other dashboard modules.

The use of Leaflet also provides interactive capabilities such as map navigation and interaction with the displayed geographic elements. This makes the map useful as part of the overall mission debriefing interface rather than simply serving as a static image.

Overall, the map implementation combines **Leaflet, JavaScript, HTML, and geographic data resources** to provide an interactive geographical component within the dashboard. It demonstrates how external visualization libraries and structured geographic data can be integrated into a web-based mission debriefing application.

7. Development Methodology

7.1. Requirement Analysis

The development of the **BVR Mission Data & Debriefing Dashboard** began with an analysis of the requirements associated with the existing mission data and the needs of the debriefing process. The primary objective of this stage was to understand what information needed to be presented, how users would interact with the dashboard, and what functionality was required to make the available mission data easier to access and review.

The requirements were identified by examining the available mission information and determining the different categories that needed to be represented within the dashboard. This resulted in the organisation of the application into multiple functional sections, including mission overview, objectives, performance, instructor summary, communication log, mission timeline, decision points, secondary tactical decisions, and replay-related information.

The data requirements were also considered during this stage. Since the mission information was stored in a PostgreSQL database, the dashboard needed a mechanism for retrieving the required records and presenting them through the web interface. This led to the requirement for a backend service capable of communicating with PostgreSQL and providing the frontend with the requested information through APIs.

User interaction requirements were identified alongside the data requirements. The dashboard needed to support navigation between different sections, dynamic display of mission information, editing of selected fields, map-based visualization, and export of mission data in CSV format. These requirements were considered while planning the structure and functionality of the application.

The technical requirements were then mapped to appropriate technologies. HTML, CSS, and JavaScript were selected for the frontend, while Node.js and Express.js were used for the backend. PostgreSQL was used for persistent data storage, and Leaflet was incorporated for map-based visualization.

The requirement analysis therefore established the functional and technical foundation for the project. It provided the basis for designing the system architecture, organising the dashboard modules, defining the backend APIs, and determining how the frontend would communicate with the database.

7.2. System Design

Following the requirement analysis, the next stage of development involved designing the overall structure of the **BVR Mission Data & Debriefing Dashboard**. The system was designed using a client-server approach in which the frontend, backend, and database perform separate but interconnected roles.

The frontend was designed as a collection of web pages, with each page dedicated to a particular category of mission information. This modular approach was adopted to avoid placing all mission information on a single page and to provide users with clear navigation

between different sections of the debriefing system. The common navigation and styling elements were designed to maintain consistency across the application.

The backend was designed as an intermediate layer between the frontend and PostgreSQL database. Instead of allowing the frontend to communicate directly with the database, API endpoints were defined within the Express.js server. This design provides a controlled mechanism for retrieving and updating mission information while keeping database operations within the server-side application.

The database design was based on the different categories of mission information identified during the requirement analysis. PostgreSQL was selected to provide structured and persistent storage, with the relevant mission information organised into appropriate tables. The backend APIs were then designed to retrieve the information required by individual dashboard modules.

The design also considered the interaction between the different components. A typical data retrieval operation follows the sequence of a user action, frontend API request, backend processing, database query, JSON response, and dynamic frontend rendering. Similarly, supported editing operations follow the reverse data flow by sending updated information from the frontend through the backend to the database.

The dashboard's visualization and supporting features were incorporated into the overall design rather than being treated as separate applications. Leaflet was integrated for map-based visualization, while CSV export was designed as a client-side feature for converting available mission information into a downloadable file.

The system design therefore established the overall structure and interaction model before implementation. It provided a clear separation of responsibilities between the presentation, application logic, data storage, and supporting visualization components, forming the basis for the subsequent development stages.

7.3. Database Design and Integration

The database design was carried out based on the different categories of information required for the **BVR Mission Data & Debriefing Dashboard**. Since the application works with structured mission information, **PostgreSQL** was selected as the database management system. The database provides persistent storage and allows the backend to retrieve the required information efficiently.

During the design stage, the available mission information was organised according to its purpose within the debriefing process. Separate database structures were used for different categories of information, including mission overview, performance, objectives, instructor summary, communication logs, mission timeline, decision points, and secondary tactical decisions. This organisation allows the individual dashboard modules to retrieve the information relevant to their respective functions.

The database was integrated with the backend rather than directly with the frontend. The Node.js and Express.js server acts as the communication layer between PostgreSQL and the dashboard. When a frontend request is received, the backend identifies the required database operation and executes the corresponding SQL query.

For data retrieval, the backend executes SELECT queries to obtain the required records from PostgreSQL. The returned records are then processed and sent to the frontend through the API as JSON data. The frontend uses this response to populate the appropriate dashboard elements.

The database design also supports modification of selected mission information. When a user edits a permitted field, the updated value is transmitted to the backend, where an appropriate SQL UPDATE operation is executed against the relevant record. This allows the database to remain synchronized with the changes made through the dashboard.

The integration between the database and application can be summarised as:

Dashboard

↓

API Request

↓

Node.js / Express.js

↓

SQL Query

↓

PostgreSQL

↓

Query Result

↓

JSON Response

↓

Dashboard

The database design and integration stage therefore established the data layer required by the dashboard. It ensured that mission information could be stored in a structured manner and accessed through controlled backend operations, providing the foundation for the data retrieval, display, and editing functionality implemented in later stages.

7.4. Frontend and Backend Development

The frontend and backend were developed together as interconnected components of the **BVR Mission Data & Debriefing Dashboard**. The frontend was responsible for providing the user interface and handling user interactions, while the backend provided the server-side logic and APIs required for accessing and managing the mission information stored in PostgreSQL.

The frontend was developed using **HTML, CSS, and JavaScript**. Separate HTML pages were created for the different sections of the dashboard, including mission overview, objectives, performance, instructor summary, communication log, mission timeline, decision points, secondary tactical decisions, and replay-related functionality. CSS was used to maintain a consistent layout and visual appearance across the different pages.

JavaScript was used to implement the dynamic behaviour of the dashboard. It handles user interactions, sends requests to the backend APIs, processes the JSON responses, and dynamically updates the appropriate elements of the interface. Additional frontend functionality, including supported data editing, CSV export, and map-based visualization, was also implemented using JavaScript.

The backend was developed using **Node.js and Express.js**. The server provides API endpoints through which the frontend can request mission information and submit supported updates. When a request is received, the backend processes the request and communicates with PostgreSQL using the appropriate SQL query.

For data retrieval, the backend executes SELECT queries and returns the resulting records to the frontend in JSON format. For supported editing operations, the frontend sends updated values through HTTP PUT requests, which are processed by the backend and applied to the relevant PostgreSQL records using SQL UPDATE operations.

The frontend and backend were developed and integrated incrementally. Individual dashboard pages and backend endpoints were implemented and tested as the corresponding functionality was developed. This allowed problems in the user interface, API communication, and database interaction to be identified and corrected during development rather than only after the complete application had been assembled.

The overall development flow can be represented as:

```
Frontend Development
        ↓
HTML + CSS + JavaScript
        ↓
Backend API Integration
        ↓
Node.js + Express.js
        ↓
```

PostgreSQL Database

↓

JSON Response

↓

Dynamic Frontend Rendering

Combining the frontend and backend development stages established the main application workflow and enabled the dashboard to operate as an integrated web-based system. The completed components provided the foundation for implementing the remaining visualization, editing, export, and debriefing functionality.

7.5. System Integration and Implementation

After the frontend, backend, and database components were developed, they were integrated to form the complete **BVR Mission Data & Debriefing Dashboard**. The integration stage focused on ensuring that the individual components could communicate correctly and that the different dashboard modules functioned together as a unified application.

The frontend was connected to the backend through the API endpoints implemented using Express.js. Each dashboard module requests the information required for its operation from the appropriate endpoint. The backend processes these requests and communicates with PostgreSQL to retrieve or update the corresponding mission records.

The data returned from the backend is provided to the frontend in JSON format. JavaScript processes the received data and dynamically renders it within the relevant interface components. This integration allows the dashboard to display database-driven information rather than relying on values embedded directly within the webpage.

Additional functionality was integrated into the application during this stage. The editing mechanism was connected to the backend update APIs so that changes made through the dashboard could be stored in PostgreSQL. The CSV export functionality was integrated with the displayed mission information, allowing users to generate a downloadable file from the available data. The Leaflet-based map component was also incorporated into the relevant dashboard interface.

Integration was performed incrementally to ensure that each component worked correctly with the others. Individual API requests were checked before being connected to the corresponding frontend modules, and database results were compared with the information displayed on the dashboard. This approach helped identify communication, data-formatting, and interface-related issues during development.

The overall integration process can be represented as:

Frontend Pages

↓

JavaScript Functions

↓

Express.js APIs

↓

PostgreSQL Database

↓

JSON Responses

↓

Frontend Rendering

↓

Dashboard Modules

↓

User Interaction

The integration stage resulted in a complete application in which the frontend, backend, database, and supporting features operate together through a common workflow. It established the final connection between the individual components and provided the working system that was subsequently subjected to testing and validation.

8. Testing and Validation

8.1. Testing Approach

Testing of the **BVR Mission Data & Debriefing Dashboard** was carried out to verify that the different components of the system function correctly and work together as expected. Since the application consists of frontend pages, backend APIs, a PostgreSQL database, visualization components, and data export functionality, testing was performed across these different layers.

The testing process primarily focused on verifying the complete flow of information from the database to the dashboard. The application was tested by accessing different dashboard pages, requesting mission information, and checking whether the expected records were retrieved from PostgreSQL and displayed correctly through the frontend. This helped verify the interaction between the frontend, backend, and database components.

The backend APIs were tested to ensure that the defined endpoints responded correctly to requests and returned the expected information in JSON format. Database operations were also checked to confirm that the required records could be retrieved and that supported updates were correctly reflected in the database.

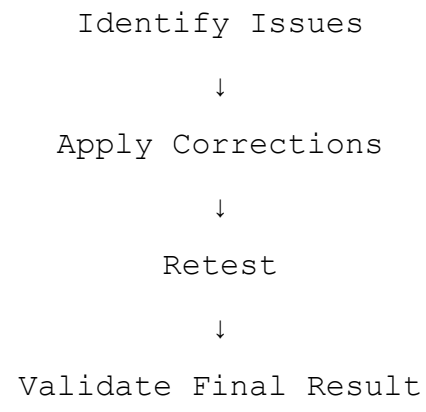
The frontend was tested by navigating through the different dashboard modules and verifying the correct rendering of mission information. Interactive features such as navigation, editable fields, map components, and other interface elements were checked to ensure that they responded correctly to user actions.

The CSV export functionality was tested by generating export files and verifying that the resulting files contained the expected mission information in a valid comma-separated format. The map component was also tested to ensure that the required geographic resources loaded correctly and that the map could be interacted with as intended.

Testing was therefore performed at both the **individual component level and the integrated system level**. This approach helped identify issues within individual components as well as problems that could occur when the frontend, backend, database, and supporting features operated together.

The testing process can be summarised as:

```
Test Requirement
        ↓
Execute Functionality
        ↓
Observe System Behaviour
        ↓
Compare with Expected Result
        ↓
```



Through this approach, the dashboard was progressively checked and refined to ensure that its major functions operated correctly and that the complete application provided a consistent experience during mission debriefing.

8.2. Frontend Testing

Frontend testing was performed to verify that the different pages and interactive components of the **BVR Mission Data & Debriefing Dashboard** function correctly from the user's perspective. The testing focused on page loading, navigation, data presentation, user interaction, and the dynamic behaviour of the dashboard.

Each major section of the dashboard was accessed individually to verify that the corresponding page loaded correctly and displayed the expected interface components. Navigation between the different sections was tested to ensure that users could move between the mission overview, objectives, performance, communication, timeline, decision, replay, and other available modules without encountering navigation errors.

The dynamic rendering of mission information was also tested. After the frontend received the JSON response from the backend, the displayed values were checked against the expected mission records to ensure that the correct information was inserted into the appropriate fields, tables, and interface components.

Interactive elements were tested by performing normal user actions such as selecting options, navigating between pages, and interacting with supported editable fields. Where editing functionality was available, changes were entered through the interface and the resulting behaviour was verified to ensure that the dashboard responded correctly.

The map component was also tested as part of the frontend. The map was checked for correct loading, display of geographic information, and basic user interaction. This helped verify that the Leaflet-based visualization was correctly integrated into the dashboard.

The CSV export interface was tested from the frontend by initiating exports and checking whether the browser correctly generated and downloaded the resulting file. The downloaded file was then opened to verify that the expected mission information had been included.

The frontend testing therefore focused on ensuring that the user-facing components of the dashboard functioned correctly, displayed the appropriate information, and responded consistently to user interactions. Any issues identified during testing were corrected and the relevant functionality was tested again to confirm the changes.

8.3. Backend and API Testing

Backend and API testing was performed to verify that the server-side components of the **BVR Mission Data & Debriefing Dashboard** correctly process requests and provide the required mission information to the frontend. The testing focused on API availability, request handling, database interaction, response format, and supported update operations.

The backend server was first tested to verify that the Node.js and Express.js application started correctly and that the defined API endpoints were accessible. Individual endpoints were then accessed with appropriate requests to confirm that they returned the expected information for the requested operation.

The API responses were checked to verify that the backend successfully retrieved the required records from PostgreSQL and returned them in **JSON format**. The returned fields and values were compared with the corresponding database records to ensure that the correct information was being transmitted to the frontend.

Database interaction through the API was also tested. Retrieval requests were used to verify that the backend could execute the required SQL queries and obtain the expected records. Supported update operations were tested by modifying permitted values through the dashboard and checking whether the corresponding changes were successfully processed by the backend and stored in PostgreSQL.

The API testing process can be represented as:

```
API Request
↓
Express.js Endpoint
↓
Request Processing
↓
SQL Query / Update
↓
PostgreSQL
↓
Result
↓
```

JSON Response

↓

Response Verification

The API responses were also observed during development using the browser's developer tools and direct API access. This provided a way to verify the actual data being returned by the server independently of the dashboard interface.

The testing confirmed that the backend provides the required communication layer between the dashboard and PostgreSQL. Successful API responses could be received by the frontend and subsequently processed for display, while supported update requests could be passed through the backend to modify the corresponding database records.

Overall, backend and API testing helped verify the reliability of the server-side communication and ensured that the required data retrieval and update operations functioned correctly as part of the complete dashboard system.

8.4. Database Testing

Database testing was performed to verify that the **PostgreSQL database** correctly stores, retrieves, and updates the mission information used by the BVR Mission Data & Debriefing Dashboard. The testing focused on the correctness of stored records and the interaction between the database and the backend application.

The database tables were checked to ensure that the required mission information was stored in the appropriate structures. Records retrieved through the backend APIs were compared with the corresponding database entries to verify that the correct values were being returned to the dashboard.

Data retrieval operations were tested using the SQL queries implemented in the backend. The results were examined to ensure that the queries returned the expected records and fields. This was particularly important for the different dashboard modules, as each module may retrieve information from a different set of mission data.

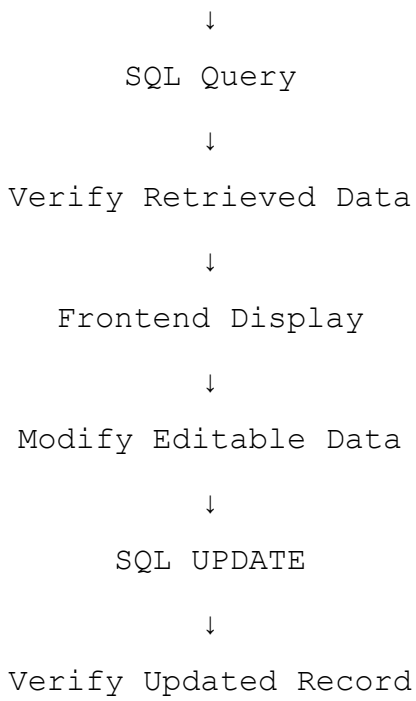
Database update operations were also tested for the editable information supported by the dashboard. After modifying a value through the user interface, the corresponding record in PostgreSQL was checked to confirm that the updated value had been stored correctly. The dashboard was subsequently refreshed or the relevant data was retrieved again to verify that the updated information was reflected correctly.

The database testing process can be summarised as:

Database Structure

↓

Insert / Existing Records



Testing also helped identify potential inconsistencies between the information stored in PostgreSQL and the information presented by the dashboard. By checking the database records independently of the frontend, it was possible to determine whether an issue originated from the stored data, the backend query, or the frontend rendering process.

The database testing confirmed that PostgreSQL functions as the persistent storage layer of the application and that the backend can successfully perform the required retrieval and update operations. This validation is important for maintaining consistency between the data stored in the database and the information presented through the dashboard.

8.5. CSV Export Testing

Testing of the CSV export functionality was performed to verify that mission information could be successfully converted into a CSV file and downloaded through the dashboard. The testing focused on the correctness of the generated file, the organisation of the exported information, and the browser download process.

The export operation was initiated through the dashboard interface, after which the application collected the relevant mission information and generated the CSV content. The resulting file was opened using a text editor to verify that the file had been created successfully and that the information was represented in comma-separated format.

The exported data was checked to ensure that the appropriate mission fields and values were included in the generated file. Different sections of mission information were also examined to verify that the exported content maintained a logical structure and that individual values were not incorrectly merged or separated.

Special characters and values containing commas were considered during testing because they can affect the structure of a CSV file if they are not handled correctly. The generated output was examined to ensure that such values remained associated with their correct fields.

The browser download mechanism was also tested to verify that selecting the export option generated the file and initiated the download correctly. The downloaded file was then opened independently of the dashboard to confirm that it remained readable and contained the expected information.

The CSV export testing process can be summarised as:

```
Select Export
      ↓
Collect Mission Data
      ↓
Generate CSV Content
      ↓
Create File Blob
      ↓
Trigger Browser Download
      ↓
Open Downloaded File
      ↓
Verify Structure & Data
```

The testing confirmed that the CSV export functionality successfully converts the mission information available through the dashboard into a reusable CSV file. The generated file can be opened independently and provides users with an additional method of retaining and accessing mission information outside the dashboard.

8.6. Map and Visualization Testing

Testing of the map and visualization components was performed to verify that the visual elements of the **BVR Mission Data & Debriefing Dashboard** were displayed correctly and functioned as intended. The testing focused on map loading, geographic data rendering, user interaction, and the consistency of the visual information presented within the dashboard.

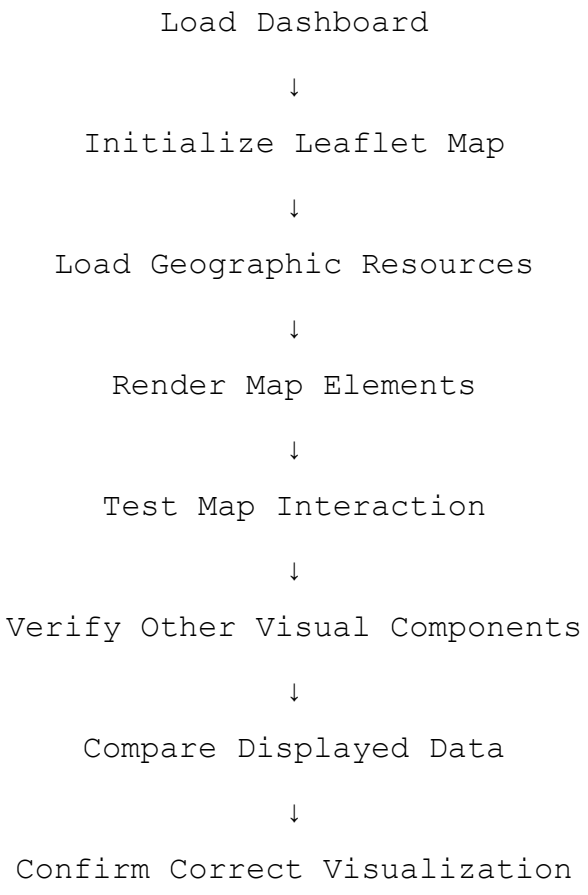
The map component was tested to ensure that the Leaflet library was loaded correctly and that the map was initialized without errors. The geographic resources used by the application were

also checked to verify that the required geographical features were rendered correctly within the map.

Basic map interactions such as zooming, panning, and navigating across the displayed geographical area were tested to ensure that the map responded correctly to user actions. The map was also checked at different zoom levels to verify that the displayed geographic information remained usable and properly positioned.

Other visual components of the dashboard, including tables, information panels, timelines, and summary elements, were reviewed to ensure that the retrieved information was presented correctly. The displayed values were compared with the corresponding data returned by the backend to identify any rendering or formatting inconsistencies.

The visualization testing process can be summarised as:



The testing confirmed that the map and associated visualization components could be integrated successfully into the dashboard and that the displayed information remained accessible and usable during normal interaction. The visualization testing also helped ensure consistency between the data received from the backend and its representation within the user interface.

8.7. Validation and Error Handling

Validation and error handling were considered during the development and testing of the **BVR Mission Data & Debriefing Dashboard** to ensure that unexpected conditions do not cause the complete application to become unusable. The testing focused on situations such as unavailable data, unsuccessful API requests, incorrect inputs, and database-related errors.

The dashboard was tested to verify that pages could handle situations where the expected mission information was unavailable or could not be retrieved. Backend API requests were also monitored to identify unsuccessful responses and communication problems between the frontend and backend.

Database-related operations were checked to ensure that failures during data retrieval or update operations could be identified without affecting unrelated parts of the application. The separation between the frontend, backend, and database layers also helped in locating the source of errors during testing.

Input validation was considered for editable fields to reduce the possibility of submitting inappropriate or incomplete values. The application was tested after modifying supported fields to ensure that valid changes could be submitted successfully and that the resulting data remained consistent with the stored database records.

Testing and validation were carried out throughout development rather than being limited to a single final testing stage. When an issue was identified, the relevant component was corrected and tested again to verify that the change resolved the problem without introducing new issues.

Overall, validation and error handling contributed to the reliability of the dashboard by providing a structured approach for identifying and responding to unexpected conditions. This helped ensure that the major dashboard functions continued to operate correctly under normal usage conditions.

9. Results and Discussion

9.1. Dashboard Result

The completed **BVR Mission Data & Debriefing Dashboard** provides a centralized web-based interface for accessing and reviewing the available mission information. The final system integrates the frontend interface, backend APIs, PostgreSQL database, visualization components, editing functionality, and data export functionality into a single application.

The dashboard successfully provides access to the different sections required for mission debriefing. Information is organised into dedicated modules so that users can review different aspects of a mission without having to interact directly with the underlying database. The consistent navigation structure also allows users to move between the different modules while remaining within the same application.

The final interface presents mission information through a combination of tables, information panels, textual fields, timelines, and map-based elements. The data displayed within these components is retrieved dynamically through the backend APIs and populated using frontend JavaScript. This demonstrates the successful integration of the application layers developed during the project.

The dashboard also provides interactive functionality beyond simple data viewing. Selected information can be edited through the interface and submitted to the backend for updating in PostgreSQL. The CSV export functionality provides an additional method of accessing the mission information by allowing the displayed data to be converted into a downloadable file.

The completed system therefore achieves the primary objective of providing a unified interface for post-mission review. Instead of requiring users to inspect raw mission records separately, the dashboard presents the available information in a structured and accessible format.

A screenshot of the completed dashboard can be included here to demonstrate the final user interface.

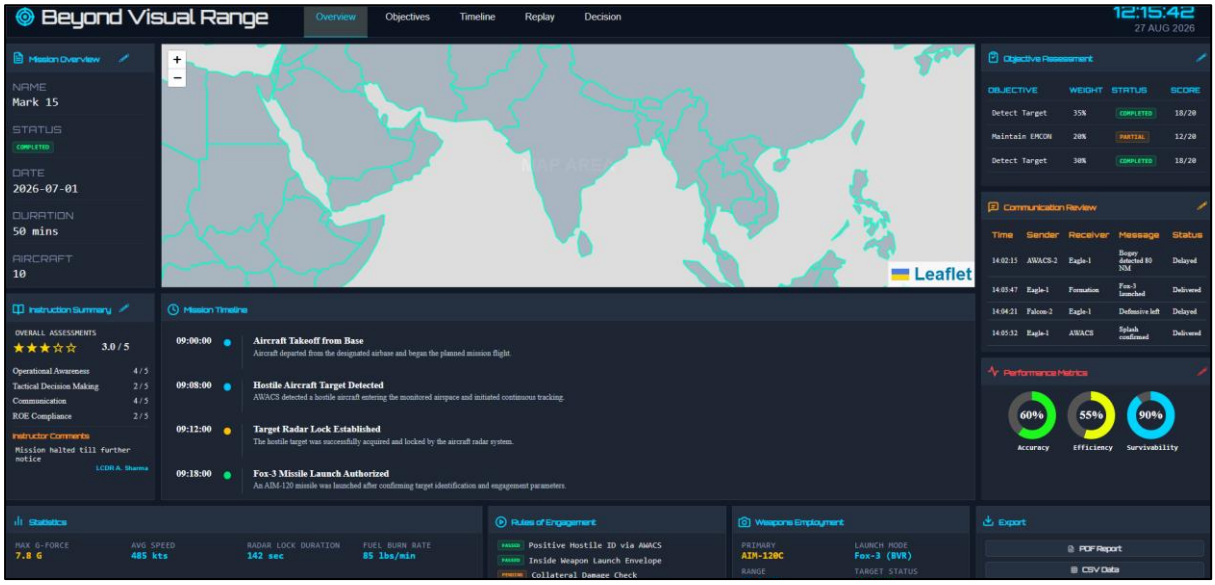


Figure 9.1: Overview Tab

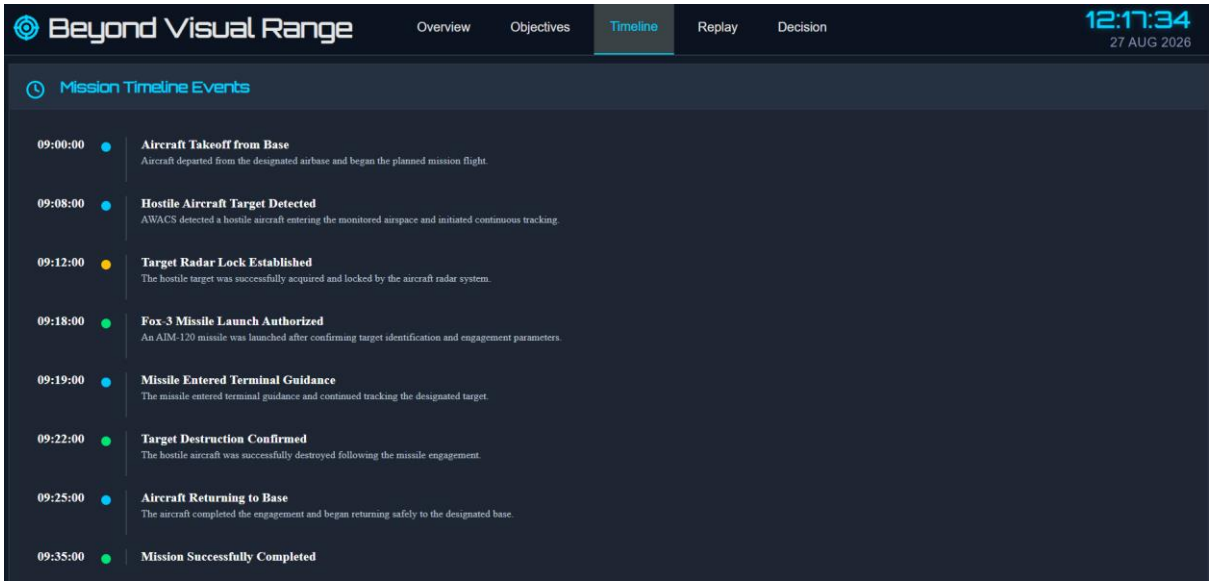


Figure 9.2: Timeline Tab

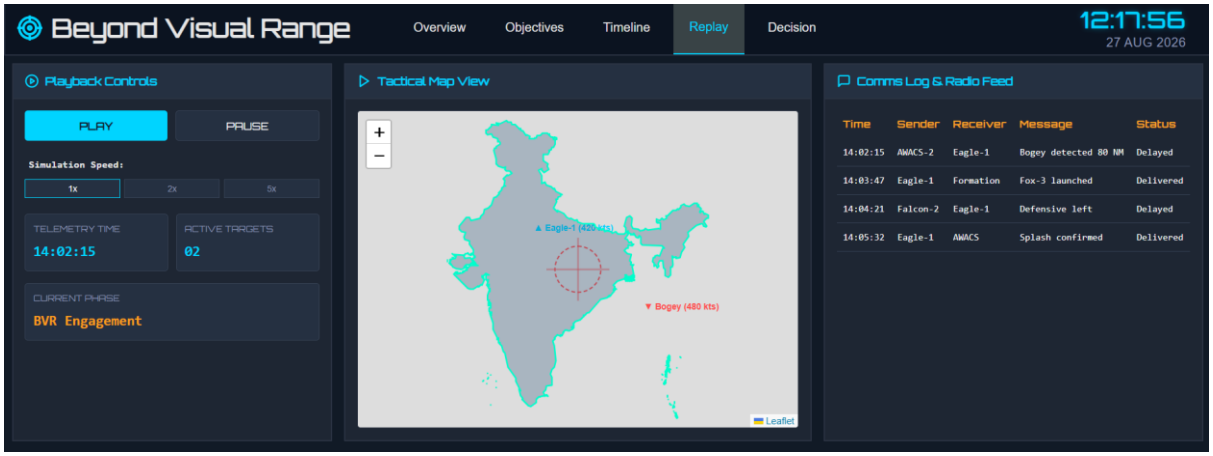


Figure 9.3: Replay Tab

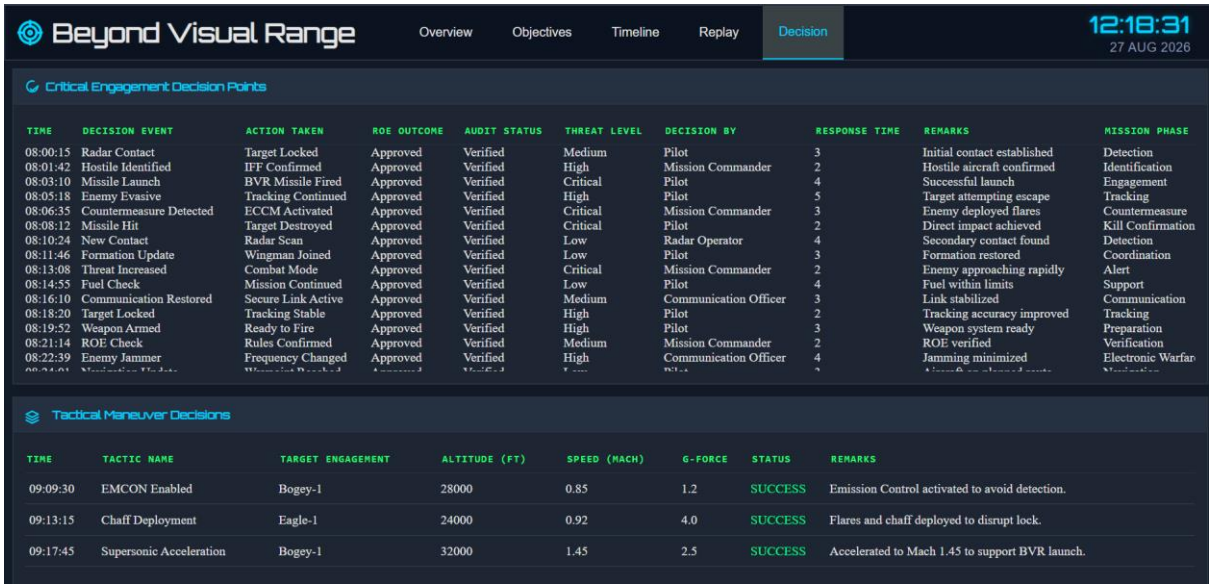


Figure 9.4: Decision Tab

The developed dashboard demonstrates the successful integration of the major system components and provides the required foundation for reviewing and managing BVR mission information through a web-based application.

9.2. Interactive Features

The completed dashboard provides several interactive features that allow users to interact with and manage the displayed mission information. These features extend the dashboard beyond static data presentation and provide a more practical interface for the post-mission debriefing process.

One of the important interactive features is the **edit functionality** provided through the pencil icons available in relevant dashboard sections. As shown in Figure 9.2, selecting the edit option opens an input interface containing the corresponding mission fields. The user can modify the permitted values and either save the changes or cancel the operation.

When the user selects **Save**, the updated information is sent from the frontend to the backend through an API request. The backend processes the request and updates the corresponding record in the PostgreSQL database. This allows the edited information to be stored rather than only changing the value temporarily on the webpage.

The dashboard also provides navigation between its different functional sections. Users can move between **Overview, Objectives, Timeline, Replay, and Decision** sections through the navigation bar. This provides access to different categories of mission information while maintaining a consistent interface.

Other interactive capabilities include the Leaflet-based map, which allows users to interact with the geographical view, and the CSV export feature, which allows mission information to be exported for external use.

The main interactive features of the dashboard include:

- **Edit Functionality:** Allows permitted mission information to be modified through the dashboard.
- **Save and Cancel Controls:** Allows the user to either confirm an edit or discard the changes.
- **Navigation:** Provides access to the different mission debriefing sections.
- **Interactive Map:** Allows users to navigate and interact with the map visualization.
- **CSV Export:** Allows the available mission information to be converted into a downloadable CSV file.

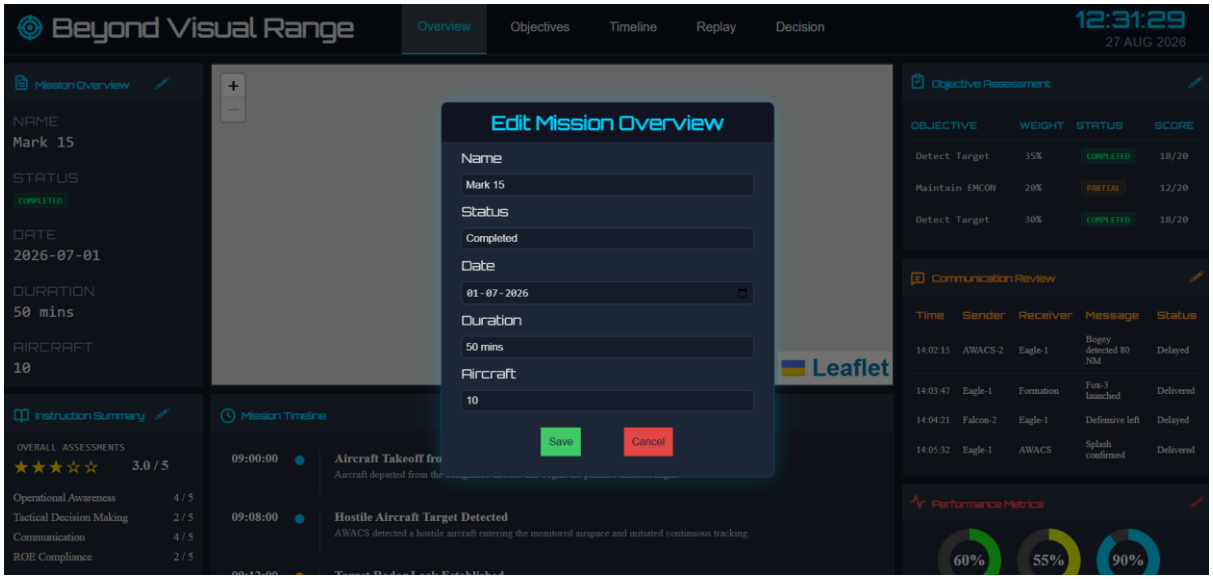


Figure 9.5: Editing interface for mission overview information

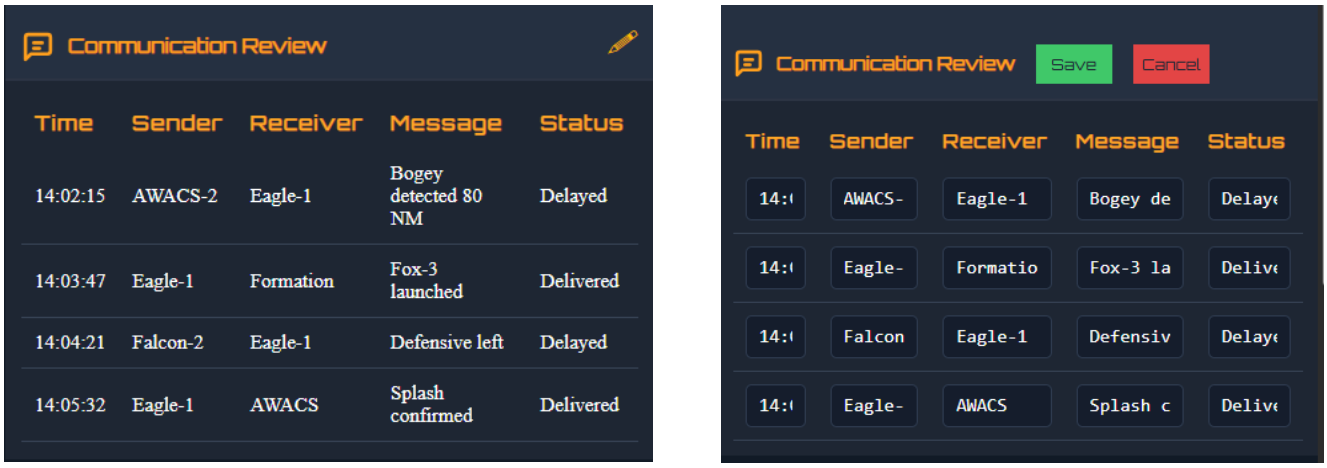


Figure 9.6: Editing interface for mission overview information

9.3. Overall System Evaluation

The completed **BVR Mission Data & Debriefing Dashboard** was evaluated as an integrated web-based system to determine whether the developed components and features met the requirements identified during the initial stages of the project. The evaluation considered the functionality of the frontend, backend, database, interactive features, visualization, and data export capabilities.

The final system successfully connects the **HTML, CSS, and JavaScript frontend** with the **Node.js and Express.js backend** and the **PostgreSQL database**. Mission information can be retrieved through the defined API endpoints and presented dynamically within the dashboard. This demonstrates that the different layers of the application operate together through the intended request-response workflow.

The evaluation also confirmed the functionality of the dashboard's major user-facing features. Users can navigate between the different debriefing sections, view mission information, edit supported fields, interact with the map, and export mission information as a CSV file. These features provide functionality beyond simple data display and make the dashboard suitable for structured post-mission review.

The system was also evaluated from the perspective of **data consistency**. Information displayed on the dashboard was compared with the corresponding information returned by the backend and stored within PostgreSQL. Similarly, edited values were checked to ensure that supported changes were successfully reflected in the database.

From an implementation perspective, the separation of the system into frontend, backend, and database layers provides a clear structure for maintenance and future development. The frontend handles presentation and interaction, the backend manages API requests and database operations, and PostgreSQL provides persistent storage for mission information.

Overall, the evaluation indicates that the developed dashboard successfully fulfils its primary objective of providing a **centralized interface for BVR mission data review and debriefing**. The integration of data retrieval, structured presentation, editing, visualization, and export functionality results in a unified system that can be used to access and manage the available mission information efficiently.

10. Limitations and Future Work

10.1.Limitations of the Present System

The **BVR Mission Data & Debriefing Dashboard** provides a centralized interface for viewing, managing, and exporting mission information; however, the present implementation has certain limitations. These limitations primarily relate to the scope of the current application, the nature of the available mission data, and the features implemented within the project.

The current system is designed primarily for **post-mission data review and debriefing**. It does not itself generate flight-dynamics data, simulate aircraft behaviour, or perform real-time mission monitoring. The dashboard depends on the mission information available in the connected database and presents that information through the implemented modules.

The scope of editable information is also limited to the fields for which editing functionality has been implemented. Not every piece of information displayed by the dashboard is necessarily intended to be modified through the user interface. This provides control over the data but limits the current editing functionality to the supported fields.

The current version also uses a defined set of dashboard modules and database structures. If additional categories of mission information are introduced, corresponding database structures, API endpoints, and frontend components may need to be added to support them.

Another limitation is that the current CSV export functionality is focused on exporting the mission information supported by the dashboard. More advanced export formats, customised reports, or automated report generation are outside the scope of the present implementation.

The map visualization is also dependent on the geographic resources available to the application. The current implementation provides map-based visualization as a supporting feature rather than a complete geographical analysis system.

Despite these limitations, the current implementation fulfils the primary objective of providing a centralized web-based interface for accessing and reviewing available BVR mission information. The identified limitations also provide opportunities for further development and expansion of the system.

10.2.Future Work

The current implementation provides the core functionality required for a BVR mission debriefing dashboard, but the system can be further extended with additional capabilities. Future development can focus on improving the depth of analysis, automation, visualization, and usability of the application.

One possible extension is the integration of **more detailed mission analytics**. Additional performance indicators and derived metrics could be incorporated to provide users with deeper insights into mission outcomes and support more comprehensive debriefing.

The dashboard could also be extended to support **real-time or near-real-time data integration** if a suitable live data source is made available. This would allow mission information to be updated automatically rather than relying only on information already stored in the database.

The existing visualization capabilities could be enhanced by introducing more advanced charts, graphs, and interactive map elements. These additions could provide users with alternative ways of examining mission timelines, performance information, and geographical data.

Another area for future development is **automated report generation**. The information currently available through the dashboard could be used to generate structured PDF or other report formats automatically, reducing the need for manual documentation after a mission.

The editing and database management functionality could also be expanded with more comprehensive validation, user roles, and controlled access. Different users could be provided with different permissions depending on their responsibilities within the debriefing process.

The system could further be extended to support **multiple missions and historical comparisons**, allowing users to compare information across missions and identify trends in performance or decision-making over time.

Finally, the dashboard architecture provides a foundation for integrating additional data sources and analytical tools in the future. New API endpoints, database structures, visualization components, and analytical features can be incorporated without fundamentally changing the existing frontend-backend-database architecture.

These enhancements could transform the current dashboard from a centralized mission review system into a more comprehensive platform for **mission analysis, reporting, visualization, and historical performance evaluation**.

11. Conclusion and Future Scope

11.1.Project Conclusion

The **BVR Mission Data & Debriefing Dashboard** was successfully developed as a web-based application for providing a centralized interface to access, review, and manage available BVR mission information. The project focused on transforming structured mission data into an organised and interactive dashboard that can support the post-mission debriefing process.

The system integrates a **HTML, CSS, and JavaScript frontend** with a **Node.js and Express.js backend** and a **PostgreSQL database**. This layered architecture allows the frontend to communicate with the backend through APIs, while the backend manages the retrieval and updating of information stored in the database. The use of JSON for communication enables the retrieved records to be processed efficiently by the frontend and dynamically displayed within the dashboard.

Multiple functional modules were implemented to organise different categories of mission information. These include mission overview, objectives, performance, instructor summary, communication log, mission timeline, decision points, secondary tactical decisions, and replay-related functionality. The system also incorporates interactive features such as editing of supported information, map-based visualization using Leaflet, and CSV export.

The project also demonstrated the integration of different technologies into a single functional application. The database provides persistent storage, the backend provides controlled access to the stored information, and the frontend provides the user-facing interface through which the information can be reviewed and interacted with.

Testing and validation of the individual components and the integrated system confirmed that the major functions operate as intended. Mission information could be retrieved from PostgreSQL through the API and displayed on the dashboard, supported values could be updated, map-based components could be accessed, and mission information could be exported into CSV format.

Overall, the project achieved its primary objective of developing a **centralized BVR mission data and debriefing dashboard**. The completed system provides a structured and accessible environment for post-mission information review while establishing a technical foundation that can be extended with additional analytical, visualization, reporting, and data-management capabilities in the future.

11.2.Learning Outcomes

The development of the **BVR Mission Data & Debriefing Dashboard** provided practical experience in designing and developing a complete web-based application. The project helped in understanding how different software components work together to create a functional, data-driven system.

A major learning outcome was gaining practical knowledge of **frontend development** using HTML, CSS, and JavaScript. The project provided experience in creating structured web

pages, designing user interfaces, implementing navigation, handling user interactions, and dynamically displaying information received from the backend.

The project also provided practical understanding of **Node.js and Express.js** for backend development. Through the implementation of API endpoints, the development process demonstrated how a server receives requests, performs the required processing, communicates with a database, and returns information to the frontend.

Working with **PostgreSQL** provided experience in relational database management and SQL-based data operations. This included understanding how structured mission information can be stored in database tables and how SQL queries such as SELECT and UPDATE can be used to retrieve and modify records.

Another important learning outcome was understanding **API communication and JSON**. The project demonstrated how data can be transferred between the backend and frontend through HTTP requests and how JSON responses can be received and processed as JavaScript objects and arrays.

The project also provided experience with additional web technologies and concepts, including **Leaflet-based map visualization, GeoJSON data, CSV generation, browser download mechanisms, and Blob objects**. These features helped demonstrate how different tools and browser capabilities can be integrated into a single application.

Overall, the project provided practical exposure to the complete development lifecycle of a web-based application, from requirement analysis and system design to implementation, integration, testing, and final evaluation.

11.3.Final Remarks

The **BVR Mission Data & Debriefing Dashboard** demonstrates the practical application of modern web development technologies to create a structured platform for post-mission data review. By integrating a responsive frontend, backend APIs, PostgreSQL database, interactive visualization, editing functionality, and CSV export, the project brings multiple aspects of mission information together within a single application.

The project also establishes a foundation that can be extended according to future requirements. Additional mission data, analytical features, visualization methods, reporting capabilities, and database functionality can be incorporated while retaining the existing modular architecture.

The development experience provided valuable practical exposure to the complete process of building a data-driven web application. The knowledge gained through requirement analysis, system design, implementation, database integration, API development, testing, and debugging provides a strong foundation for undertaking more complex software development projects in the future.

12. Conclusion

12.1.Project References

The following official documentation and technical resources were referred to during the development and implementation of the **BVR Mission Data & Debriefing Dashboard**:

1. **Node.js Documentation** — Official documentation for the Node.js JavaScript runtime, including server-side JavaScript, HTTP functionality, modules, and runtime APIs.

2. **Express.js Documentation** — Official documentation for the Express.js web framework used for creating the backend server, routes, APIs, and middleware.

3. **PostgreSQL Documentation** — Official PostgreSQL documentation referred to for relational database concepts, SQL queries, data manipulation, and database operations.

4. **MDN Web Docs — JavaScript** — Reference material for JavaScript concepts, objects, arrays, DOM manipulation, and client-side scripting used in the dashboard.

5. **MDN Web Docs — Fetch API** — Reference material for making HTTP requests from the frontend and receiving responses from backend APIs.

6. **MDN Web Docs — JSON** — Reference material for understanding JSON as a data-interchange format and parsing JSON responses into JavaScript objects and arrays.

7. **MDN Web Docs — Blob and Object URLs** — Reference material for the Blob and object URL mechanisms used in the browser-based CSV download implementation.

8. **Leaflet Documentation** — Documentation for the Leaflet JavaScript library used to implement interactive map visualization within the dashboard.

12.2. Bibliography

[1] D. Flanagan, “JavaScript: The Definitive Guide,” 7th ed., O’Reilly Media, 2020.

[2] J. Duckett, “HTML and CSS: Design and Build Websites,” Wiley, 2011.

[3] J. Duckett, “JavaScript and JQuery: Interactive Front-End Web Development,” Wiley, 2014.

[4] R. Elmasri and S. B. Navathe, “Fundamentals of Database Systems,” 7th ed., Pearson, 2016.

[5] A. Silberschatz, H. Korth, and S. Sudarshan, “Database System Concepts,” 7th ed., McGraw-Hill, 2019.

[6] R. S. Pressman and B. R. Maxim, “Software Engineering: A Practitioner’s Approach,” 9th ed., McGraw-Hill Education, 2019.

[7] I. Sommerville, “Software Engineering,” 10th ed., Pearson, 2016.

[8] R. Nixon, “Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5,” 6th ed., O’Reilly Media, 2021.

[9] M. Fowler, “UML Distilled: A Brief Guide to the Standard Object Modeling Language,” 3rd ed., Addison-Wesley Professional, 2004.